ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

inSight/am®
Programmer's
Reference Guide

DSM-00188-02 A00

© 1995 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
2.0	November 1994

inSight/am Programmer's Reference Guide

To re-order this document, request part number DSO-00180-00

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document was written by Peter Winter, Sandi Tomblin, and Bob Shelton.
Illustrations by Peter Winter.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VER-A-TEL
VIDEOTRAX			

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
DART	ESP	MULTI	*in*Sight/am
*in*Front/am			

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

TABLE OF CONTENTS

CHAPTER ONE - INTRODUCTION 1-1

ABOUT THIS BOOK 1-1
WHAT IS *IN*SIGHT? 1-1
REQUIREMENTS 1-1
GRAPHICS CONVENTIONS 1-1

CHAPTER TWO - INSTALLATION NOTES 2-1

INSTALLING THE ALPHAACCOUNTING SUBROUTINES 2-1
INSTALLING THE ALPHAACCOUNTING SUBROUTINES IN [7,6] 2-1
INSTALLING THE ALPHAACCOUNTING SUBROUTINES IN [7,60] 2-2
USING THE RIGHT TERMINAL DRIVER 2-2
FUNCTION KEY TRANSLATION TABLES 2-2

CHAPTER THREE - INSIGHT FUNCTIONAL GROUPS 3-1

FUNCTIONAL GROUPS 3-1
SETUP FUNCTIONS 3-1
ENVIRONMENT FUNCTIONS 3-2
USER INTERFACE INPUT FUNCTIONS 3-3
PROGRESS INDICATOR FUNCTIONS 3-3
PROGRAM EXECUTION FUNCTIONS 3-4
IMAGE DISPLAY FUNCTIONS 3-4
FILE I/O FUNCTIONS 3-5
*IN*SIGHT HELP SYSTEM FUNCTIONS 3-5
WINDOWS PROFILE ACCESS FUNCTIONS 3-6
DYNAMIC LINK LIBRARY FUNCTIONS 3-6
OTHER FUNCTION 3-6

CHAPTER FOUR - INSIGHT FUNCTION REFERENCE SHEETS 4-1

CALLING METHODS 4-1
ABOUT THE DIRECT CALLING METHOD 4-1
BOLD TYPE IN CODE EXAMPLES 4-2
ACCESS A PROGRESS BAR (DIRECT) 4-3
ACCPT.SBR XCALL 4-5
CAPTION (ALPHABASIC) 4-7
CHANGE CAPTION LINE (DIRECT) 4-8
CHANGE CURSOR SHAPE (DIRECT) 4-9
CLEAR'BITMAPS (ALPHABASIC) 4-10
CLEAR'SCREEN (ALPHABASIC) 4-11
CLOSE A FILE (DIRECT) 4-12

CLOSE'FILE (ALPHABASIC)....4-13
DISPLAY A LIST BOX (DIRECT)....4-14
DISPLAY A MENU (DIRECT)....4-15
DISPLAY A MESSAGE BOX (DIRECT)....4-16
DISPLAY AN EDIT TEXT BOX (DIRECT)....4-17
DISPLAY AN EXTENDED EDIT TEXT BOX (DIRECT)....4-18
EDIT & XEDIT (ALPHABASIC)....4-19
EXIT TERMINAL EMULATION WINDOW (DIRECT)....4-21
FREE A DLL (DIRECT)....4-22
FSTRIP.SBR (XCALL)....4-23
GET A FILENAME (DIRECT)....4-24
GET'FILENAME (ALPHABASIC)....4-25
IEXIT (AMOS)....4-27
INITIATE ACTION AGAINST A FILE (DIRECT)....4-28
IVT'INFO (ALPHABASIC)....4-30
IVT'LAUNCH (ALPHABASIC)....4-31
IVTCAP (AMOS)....4-32
IVTERM'CLOSE (ALPHABASIC)....4-33
IVTERM'INIT (ALPHABASIC)....4-34
IVTVER (AMOS)....4-35
JOBDAT.SBR XCALL....4-36
LAUNCH A WORKSTATION PROGRAM (DIRECT)....4-39
LAUNCH WINDOWS HELP SYSTEM (DIRECT)....4-40
LAUNCH'HELP (ALPHABASIC)....4-41
LIST (ALPHABASIC)....4-43
LOAD A DLL (DIRECT)....4-45
MAKE A CALL INTO A DLL (DIRECT)....4-46
MENU (ALPHABASIC)....4-47
MESSAGE'BOX (ALPHABASIC)....4-49
MODIFY'BITMAP (ALPHABASIC)....4-51
MODIFY BITMAP IMAGE (DIRECT)....4-53
OPEN'FILE (ALPHABASIC)....4-55
PROGRESS (ALPHABASIC)....4-57
READ OR RE-READ A RECORD (DIRECT)....4-59
READ'FILE (ALPHABASIC)....4-61
READ'PROFILE'BOOL (ALPHABASIC)....4-62
READ'PROFILE'INT (ALPHABASIC)....4-63
READ'PROFILE'STRING (ALPHABASIC)....4-64
READ OR WRITE DATA FROM A PRIVATE PROFILE FILE (DIRECT)....4-65
RETRIEVE WORKSTATION INFORMATION (DIRECT)....4-67
SHOW'BITMAP (ALPHABASIC)....4-68
SHOW'BITMAP'FROM'LIST (ALPHABASIC)....4-71
SHOW BITMAP IMAGE (DIRECT)....4-73
WINFLG.SBR XCALL....4-75
WRITE OR REWRITE A RECORD (DIRECT)....4-77
WRITE'FILE (ALPHABASIC)....4-79
WRITE'PROFILE'BOOL (ALPHABASIC)....4-80
WRITE'PROFILE'INT (ALPHABASIC)....4-81

WRITE'PROFILE'STRING (ALPHABASIC) 4-82

CHAPTER FIVE - THE ALPHAACCOUNTING SUBROUTINES 5-1

ANYCN CHECK FOR CHANGE 5-3
MESAG DISPLAY STANDARD ERROR MESSAGES 5-4
MMENU DISPLAY STANDARD MASTER MENU 5-5
TMENU DISPLAY STANDARD TRANSACTION MENU 5-6

CHAPTER SIX - PROGRAMMING AND INTEGRATION METHODS 6-1

PHILOSOPHY 6-1
THE ALPHALAN ENVIRONMENT 6-1
MODIFYING YOUR PROGRAM 6-3
SETUP 6-3
CAPTION 6-4
MENU 6-4
CARRIAGE RETURN TO CONTINUE 6-6
COLORIZATION 6-7
PROGRESS INDICATORS 6-7
LIST BOXES 6-8
MOUSE ENABLING YOUR SCREENS 6-10
THE MOUSE AND INFLD 6-14
SIMPLE IMAGING 6-15
HELP SYSTEM 6-19
LAUNCHING WINDOWS PROGRAMS 6-21
PRIVATE PROFILE ACCESS 6-22
FILE I/O 6-23

CHAPTER SEVEN - WRITING AN INSIGHT DLL CALL 7-1

DEFINING TYPES 7-1
DECLARING THE FUNCTION 7-1
SENDING DATA BACK TO THE HOST PROGRAM 7-2
PARAMETERS 7-2
RETURN VALUE 7-2
EXAMPLE 7-2
SAMPLE DLL DEFINITION FILE 7-3
SAMPLE DLL HEADER FILE 7-3
SAMPLE DLL PROGRAM FILE 7-3

APPENDIX A - INSIGHT SOFTWARE: FILE LIST A-1

APPENDIX B - MODIFYING TERMINAL DRIVERS B-1

INDEX I-1

CHAPTER ONE
INTRODUCTION

*in*Sight™ for AMOS® combines with terminal emulation software like AlphaLAN® to let you use *in*Sight-equipped workstations as terminals on your AMOS host computer. With *in*Sight, your AMOS applications can use icons, button menus, dialog boxes, and many of the other **G**raphical **U**ser **I**nterface (GUI) features that make Microsoft® Windows™ so popular.

The AMOS portion of the *in*Sight software consists of the *in*Sight control library and AlphaBASIC subroutines which, combined with the *in*Sight DLL (Dynamic Linked Library) on the workstation, let your programs access the GUI features available. These routines also simulate GUI features when used on a character-oriented terminal.

ABOUT THIS BOOK

This manual has all the information you need to update your AMOS applications to use the *in*Sight GUI features available. Besides this introduction, it contains the following parts:

- Chapter 2 contains supplemental installation information for the AMOS host computer.
- Chapter 3 categorizes *in*Sight's commands into functional groups for easy reference.
- Chapter 4 is an alphabetic reference section for all of *in*Sight's commands.
- Chapter 5 covers the AlphaACCOUNTING AlphaBASIC XCALLs included with *in*Sight for AlphaACCOUNTING-based applications.
- Chapter 6 describes *in*Sight's programming philosophy and provides you with programming tips for integrating *in*Sight into your application.
- Chapter 7 gives some rules you need to follow if you write a Windows DLL (Dynamic Link Library) routine to be called by *in*Sight.
- Appendix A contains a chart of the *in*Sight file names and their disk locations.
- Appendix B describes how you can modify your terminal driver programs to support *in*Sight's GUI features.

WHAT IS *in*SIGHT?

*in*Sight is Alpha Micro's open system architecture that combines the advantages of central host-based computing, local desktop processing, and high-speed networking. More than a single

hardware or software product, *in*Sight is a broad-based philosophy whose realization seamlessly combines host-based and workstation-based computing to best effect.

*in*Sight is a quantum leap in terms of features, functionality and power you can deliver to your users. It is a unifying, cross-platform strategy that combines your application with *in*Sight products and features into a coherent, relevant and powerful whole. *in*Sight will provide you the computing platform to take your business and your user base through this decade and into the next.

Much has been made of the battle between centralized host-based computing and desktop, network-based computing. By taking the best features of both approaches, *in*Sight provides an application development and execution environment which cannot be matched by any solution that relies on a single approach.

But *in*Sight's greatest advantage is that it gives you complete freedom to mix and match host and desktop resources. You can create the best environment for your application's and user's needs. Through a cooperative client-server relationship, computing tasks can be shared between the host and the workstation. Properly structured, a program that ran only on a host using character-based terminals now also runs on the workstation, taking advantage of the workstation's GUI and computing capabilities. The same source and executable code can support any combination of character-based terminals and workstations attached to the same host.

Using the workstation's resources can give your existing applications a modern GUI look and feel which today's computer user is comfortable with. That user is very likely to demand such a GUI in the application's specification. *in*Sight can provide you that necessary functionality—today!

REQUIREMENTS

To use the *in*Sight software, you need:

1. An AMOS host computer running AMOS 1.4 or later or AMOS 2.2 or later.
2. One or more PC-compatible workstations with Microsoft Windows 3.1 or later and the *in*Sight Tool Kit, a Windows Dynamic Linked Library (DLL), installed. The workstation must also have an *in*Sight compatible terminal emulator, such as AlphaLAN/FW.
3. A connection between the workstation and host. This can be either a serial connection (such as an RS-232 cable) or an Ethernet connection.
4. If you are using Ethernet, the host must have AlphaNET 2.2A or later and each workstation must have the AlphaNET PC support software installed. To use type 2 virtual terminal connections with AlphaNET, you must have AMOS 2.2 or later.

GRAPHICS CONVENTIONS

This manual conforms to the other Alpha Micro publications in its use of a standard set of graphics conventions. We hope these graphics simplify our examples and make them easier for you to use.

Throughout this book, we tell you what actions to perform to make *in*Sight do what you want it to. To make these instructions easier to understand, we use some special typefaces and other conventions to let you know what we're talking about. These are:

Convention	Meaning
Cancel	When we talk about one of the "buttons" on a screen, the button name is in italics.
File Menu	When we refer to a Windows menu name, or one of the choices on a Windows menu bar or drop-down menu, we start each word of the name with a capital letter.
`text`	We use this typeface to show program syntax and for programming examples.
name	In explanations of program format, the first reference to a variable name or another part of the example is in this bold, italic text.
	When you see this symbol, pay careful attention. It indicates a warning—often something you either must do or must not do.
	When you see this symbol, there is a helpful bit of information that points out a short cut which may save you time or trouble.
	This symbol marks information you need to keep in mind while you are following a set of instructions and information related to, but not actually part of, the current topic.
RETURN	When we tell you to press a key on your keyboard, we use a keycap symbol with the name of the key inside. If you need to press two keys at once, the key names are separated by a plus sign. For example, ALT + F4.

CHAPTER TWO
INSTALLATION NOTES

How to install the *in*Sight software is described in the *Release Notes* for your version of *in*Sight. This chapter contains supplementary installation information about these two areas:

- Renaming the included versions of the AlphaACCOUNTING subroutines so your application can use them.
- Making sure the *in*Sight workstations use the correct terminal driver and creating function key translation tables for that terminal driver if necessary.

INSTALLING THE ALPHAACCOUNTING SUBROUTINES

The *in*Sight AlphaACCOUNTING assembly language subroutines have the extension .VBR instead of .SBR. Before you can use them, you need to rename them. For most computers, follow the first procedure. However, if your programs specifically load the AlphaACCOUNTING subroutines from DSK0:[7,60], not from BAS:, use the second procedure.

Installing the AlphaACCOUNTING Subroutines in [7,6]

1. At the AMOS prompt, type:

   LOG BAS: RETURN

2. To keep the standard versions of the AlphaACCOUNTING subroutines from being overwritten by the new ones that have been modified for *in*Sight, type:

   If you've previously installed an earlier version of *in*Sight, skip this step. If you enter these commands, you'll be copying over your saved versions of the subroutines with the earlier *in*Sight versions.

   RENAME *.SAV=ANYCN.SBR/D RETURN
   RENAME *.SAV=MESAG.SBR/D RETURN
   RENAME *.SAV=MMENU.SBR/D RETURN
   RENAME *.SAV=TMENU.SBR/D RETURN

3. If no files are renamed, the old-style subroutines are not in BAS:—no harm is done.

4. Make a copy of the *in*Sight subroutines with the .SBR extension so your applications can use them:

   COPY *.SBR=ANYCN.VBR RETURN
   COPY *.SBR=MESAG.VBR RETURN
   COPY *.SBR=MMENU.VBR RETURN
   COPY *.SBR=TMENU.VBR RETURN

Installing the AlphaACCOUNTING Subroutines in [7,60]

If your programs load the AlphaACCOUNTING subroutines from DSK0:[7,60], use this procedure instead of the one above:

1. At the AMOS prompt type:

   LOG DSK0:[7,60]RETURN

2. To keep the standard versions of the AlphaACCOUNTING subroutines from being overwritten by the new ones that have been modified for *in*Sight, type:

   If you've previously installed an earlier version of *in*Sight, skip this step. If you enter these commands, you'll be copying over your saved versions of the subroutines with the earlier *in*Sight versions.

   RENAME *.SAV=ANYCN.SBR/D RETURN
   RENAME *.SAV=MESAG.SBR/D RETURN
   RENAME *.SAV=MMENU.SBR/D RETURN
   RENAME *.SAV=TMENU.SBR/D RETURN

   If no files are renamed, the old-style subroutines are not in DSK0:[7,60]—you should be installing the routines in [7,6], as described above.

3. Make a copy of the *in*Sight subroutines with the .SBR extension so your applications can use them:

   COPY *.SBR=ANYCN.VBR[7,6]RETURN
   COPY *.SBR=MESAG.VBR[7,6]RETURN
   COPY *.SBR=MMENU.VBR[7,6]RETURN
   COPY *.SBR=TMENU.VBR[7,6]RETURN

USING THE RIGHT TERMINAL DRIVER

Any workstation on which you want to be able to use the *in*Sight functions must use the PCI terminal driver. Change the terminal definition lines in your system initialization command file so the workstations use this driver. The PCI driver is based on the PC driver used by AlphaLAN for its native mode emulation, with *in*Sight support added.

As always, modify a copy of your .INI file, not the original!

Function Key Translation Tables

As always when using a new terminal driver, you should make sure all the necessary function key translation tables exist for the PCI driver. Since the PCI driver is very similar to the PC driver, copy those translation tables if they exist. If they don’t, you can copy the translation tables for the AM-62A terminal.

CHAPTER THREE
*in*SIGHT FUNCTIONAL GROUPS

This chapter defines the general categories of *in*Sight commands and functions, lists their names, and provides a brief description of each. The next chapter gives detailed information about each, in quick reference sheet format.

There are four ways to invoke *in*Sight functions: an AlphaBASIC subroutine, an AMOS command, an AlphaBASIC XCALL, or by calling the *in*Sight function directly. For more information about these methods, see the introduction to Chapter 4.

FUNCTIONAL GROUPS

The *in*Sight functions are divided into these categories: Setup, Environment, User Interface Input, Progress Indicator, Program Execution, Image Display, File I/O, Help System, Windows Private Profile File Access, Dynamic Link Library, and Other.

Setup Functions

Routines dealing with setting up and terminating the inSight environment are:

FUNCTION NAME	METHOD/PURPOSE
Exit	Direct Exits the terminal emulation window
IEXIT	AMOS Exits the terminal emulation window
IVTERM'CLOSE	AlphaBASIC Used when the user completely exits the application; causes exit from and complete close of the terminal emulation window.
IVTERM'INIT	AlphaBASIC Used prior to any other *in*Sight calls; sets up variables about the current job.

Environment Functions

Functions dealing with workstation display and environment—both setting it up and retrieving information about it—are categorized as Environment Functions:

FUNCTION NAME	METHOD/PURPOSE
CAPTION	AlphaBASIC Modifies the terminal emulation window's caption line.
Change caption line	Direct Modifies the terminal emulation window's caption line.
Change cursor shape	Direct Changes the shape of the mouse cursor.
CLEAR'SCREEN	AlphaBASIC Clears the screen and displays user information on the top two lines. Use this in place of TAB(-1,0) to create consistent screen processing—each screen will have a similar look and feel.
IVT'INFO	AlphaBASIC Retrieves size information about the workstation screen and terminal emulation window.
IVTCAP	AMOS Modifies the terminal emulation window's caption line.
IVTVER	AMOS Displays the current *in*Sight version on the workstation.
JOBDAT.SBR	XCALL Subroutine to return the job information from an AlphaBASIC program.
Retrieve workstation information	Direct Retrieves information about the currently attached workstation.
WINFLG.SBR	XCALL Subroutine to return *in*Sight flags from an AlphaBASIC program.

User Interface Input Functions

User Interface Input Functions deal with receiving information from the user. These typically provide sophisticated input methods depending on the type of information being asked for:

FUNCTION NAME	METHOD/PURPOSE
ACCPT.SBR	XCALL Subroutine to accept a single character from the keyboard **without echo.**
Display a list box	Direct Displays a scrolling Windows list box.
Display a menu	Direct Displays a menu of items from which the user selects.
Display a message box	Direct Displays a standard Windows message box, with the text and buttons defined by the host program.
Display an edit text box	Direct Displays a Windows edit box in which the user can make or change an entry.
Display extended edit text box	Direct An extended version of the preceding function.
EDIT	AlphaBASIC Displays and edits a single text line within a window.
LIST	AlphaBASIC Displays an item list for user selection.
MENU	AlphaBASIC Displays a menu of items from which the user selects.
MESSAGE'BOX	AlphaBASIC Displays a standard message box on the screen. Message text can be one to twenty lines long. Button options are also provided: OK; CANCEL, YES, NO, etc.
XEDIT	AlphaBASIC Extended version of EDIT.

Progress Indicator Functions

These routines show a work-in-progress indicator:

FUNCTION NAME	METHOD/PURPOSE
Access a progress bar	Direct Initializes, updates, and closes up to ten simultaneous progress indicators.
PROGRESS	AlphaBASIC Displays a progress indicator giving the user an indication of how far along the procedure is.

Program Execution Functions

Routines that allows the host program to execute workstation programs:

FUNCTION NAME	METHOD/PURPOSE
IVT'LAUNCH	AlphaBASIC Launches a workstation application from the host.
Launch a workstation program	Direct Launches a workstation application from the host.

Image Display Functions

These routines display and manipulate images on the workstation:

FUNCTION NAME	METHOD/PURPOSE
CLEAR'BITMAPS	AlphaBASIC Clears bitmap images selected via SHOW'BITMAP'FROM'LIST from the screen.
Modify bitmap image	Direct Changes the characteristics of an existing bitmap image window.
MODIFY'BITMAP	AlphaBASIC Changes display characteristics of a bitmap image.
SHOW'BITMAP	AlphaBASIC Displays a bitmap image on the workstation.
SHOW'BITMAP'FROM'LIST	AlphaBASIC Presents user with a preloaded list of bitmap images from which to select.
Show bitmap image	Direct Displays a bitmap image on the workstation.

File I/O Functions

These routines allow simple file access to and from the workstation:

FUNCTION NAME	METHOD/PURPOSE
Close a file	Direct Closes an opened file.
CLOSE'FILE	AlphaBASIC Closes an opened file.
Get a filename	Direct Lets user select a file name using the file selector native to the workstation.
GET'FILENAME	AlphaBASIC Lets user select a file name using the file selector native to the workstation.
Initiate action against a file	Direct Opens, looks up, or deletes a file.
OPEN'FILE	AlphaBASIC Opens, looks up, or deletes a file.
Read or re-read a record	Direct Reads a record from a file.
READ'FILE	AlphaBASIC Reads a record from a file.
Write or re-write a file	Direct Writes a record to a file.
WRITE'FILE	AlphaBASIC Writes a record to a file.

*in*Sight Help System Functions

These routines access *in*Sight's help system or other Windows help files:

FUNCTION NAME	METHOD/PURPOSE
LAUNCH'HELP	AlphaBASIC Launches *in*Sight's help system.
Launch Windows Help System	Direct Launches a Windows help file.

Windows Profile Access Functions

These routines allow reading and writing items in a Windows Private Profile file. Refer to Chapter 6 for more information on Private Profile files.

FUNCTION NAME	METHOD/PURPOSE
Read or write data in a private profile file	Direct Reads or writes data from/to a private profile file on the PC, using standard Windows syntax.
read'profile'string	AlphaBASIC Reads a string from a private profile file.
read'profile'int	AlphaBASIC Reads an integer from a private profile file.
read'profile'bool	AlphaBASIC Reads a Boolean value from a private profile file.
write'profile'string	AlphaBASIC Writes a string to a private profile file.
write'profile'int	AlphaBASIC Writes an integer to a private profile file.
write'profile'bool	AlphaBASIC Writes a Boolean value to a private profile file.

Dynamic Link Library Functions

These functions let you load a Windows DLL, call functions from it, and unload it when you are finished:

FUNCTION NAME	METHOD/PURPOSE
Free a DLL	Direct Frees a previously loaded DLL.
Load a DLL	Direct. Loads a DLL so you can call functions from it.
Make a call into DLL	Direct Calls a function within a loaded DLL. You can include data with the call.

Other Function

This category of commands is a catchall:

FUNCTION NAME	METHOD/PURPOSE
FSTRIP.SBR	XCALL Subroutine to strip all spaces from both the front and rear of a string.

CHAPTER FOUR
*in*Sight FUNCTION REFERENCE SHEETS

The following pages describe the set of functions that make up *in*Sight.

The updated AlphaACCOUNTING XCALLs included with *in*Sight are described in Chapter 5.

CALLING METHODS

The description of each function includes the calling method listed to the right of the function name. These are the designations we use:

CALLING METHOD	DESCRIPTION
AMOS	This is an AMOS level command.
AlphaBASIC	This is an AlphaBASIC subroutine. These functions are defined in INCLUDE files that are part of the *in*Sight AMOS tool kit.
Direct	The specific escape and command sequence needed to initiate an *in*Sight function on the workstation. These commands are useful to non-AlphaBASIC programs or on non-AMOS platforms. This is the lowest level of *in*Sight programming.
XCALL	These are AlphaBASIC XCALLs required to support the AlphaBASIC *in*Sight tool kit. Their functional descriptions are provided for your information.

ABOUT THE DIRECT CALLING METHOD

The following information applies to *in*Sight functions categorized as Direct:

- All examples use the AlphaBASIC method for simplicity. Translating that protocol may be necessary for your native language.
- The TAB(-1,192) command is the required lead-in for **every** *in*Sight function. This is the same as TCRT (-1,192). For non-AMOS systems, use the following translation:

  ESC S Terminal emulation running in native mode.
  ESC D All other terminal emulations.

- The [DEL] key is used as an argument/function terminator. This key is equivalent to an ASCII 127. Where specified use the appropriate output for your language (for Assembly language, you could use MOV #127.,D1[RETURN] TTY[RETURN]). In our AlphaBASIC examples, we set the DEL variable equal to CHR$(127), then use it in the *in*Sight function.

BOLD TYPE IN CODE EXAMPLES

In the samples of program code included on the function reference sheets, all items in **bold** type are defined in the *in*Sight include files.

Access a progress bar (Direct)

This function initializes, updates and closes up to ten simultaneous progress indicators.

Syntax

Three calls control the progress indicator.

To initialize: `TAB(-1,192) P Whcbar space Flag Title` DEL

To update: `TAB(-1,192) P Whcbar ! Cur` DEL `Max` DEL

To close: `TAB(-1,192) P Whcbar "`

Whcbar is a single character between space (ASCII 32) and * (ASCII 41). This defines which of the ten possible indicators you are accessing.

Flag is reserved for future use. It must be a space (ASCII 32).

Title is the string defining the window caption for the progress indicator.

Cur is the ASCII numeric value of the current position within the procedure. This value must be between 0 and Max.

Max is the ASCII numeric value of the maximum position in the procedure.

Returns

None.

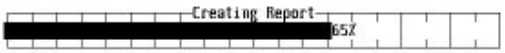
Example

The following example performs a procedure which is made up of three subprocedures. A main progress indicator tracks the completion of the total procedure; a second indicator shows the progress of each subprocedure.

```
MAP1 del    , S, 1, CHR$(127)
MAP1 RUNS(3), F, 6
MAP1 TOTAL  , F, 6
MAP1 CNT    , F, 6, 0
 RUNS(1)= 50
 RUNS(2)= 75
 RUNS(3)= 150
 TOTAL= 275
PRINT TAB(-1,192);"P   Main Progress";del;
FOR X = 1 TO 3
      PRINT TAB(-1,192);"P!  Sub Progress";del;
      FOR Y = 1 TO RUNS(X)
            PRINT TAB(-1,192);"P!!";STR(Y);del;STR(RUNS(X));del;
```

```
            CNT= CNT + 1
            PRINT TAB(-1,192);"P !";STR(CNT);del;STR(TOTAL);del;
      NEXT Y
      PRINT TAB(-1,192);"P!";CHR$(34);
NEXT X
PRINT TAB(-1,192);"P ";CHR$(34);
```

Here are samples of a progress indicator on an *in*Sight workstation and a character terminal:



Synonym

PROGRESS (AlphaBASIC)

ACCPT.SBR XCALL

This user interface routine is an AlphaBASIC subroutine designed to wait for a single character from the keyboard. Unlike ACCEPT.SBR, which echoes the character automatically, ACCPT simply returns the character to the calling program.

ACCPT.SBR is useful for receiving raw characters from the keyboard. The character returned can be anything from the entire ASCII set depending on the key the user presses. Normally this subroutine waits for the user to press a key prior to returning to the calling program. If you specify a second argument, as described below, ACCPT **always** returns immediately. If characters are present, the first character in the buffer is returned; otherwise ACCPT returns a 0, indicating no input is pending.

ACCPT.SBR works with all versions of AlphaBASIC. It is reentrant and reusable and may be loaded into either system or user memory.

Prerequisite

`XCALL NOECHO` (or equivalent).

Syntax

```
XCALL ACCPT, inchr {,1}
```

inchr is any valid AlphaBASIC six-byte floating point variable. The ASCII value of the input character is returned in this variable. It can be in the range of 1 to 255 (0 - 255 if the second argument is defined.) It cannot be a literal value.

1 is the optional second argument and can be of any type, literal or variable. The presence of a second argument tells ACCPT **not** to wait for input and to return immediately with a 0 in inchr if no data is present.

To change the numeric value of the returned character to a string character, use AlphaBASIC's CHR$ function. This is most useful on printable ASCII characters (32 - 126.)

Examples

To build an input string using ACCPT you could do the following:

```
MAP1        instr  ,  S     , 30
MAP1        inchr  ,  F     ,  6
XCALL       NOECHO

get'string:
instr       = ""
gs001:
  XCALL     ACCPT,  inchr
  IF        inchr  = 13  RETURN
  IF        (inchr = 127) AND (LEN(instr) > 0)  &
            instr  = instr(1,-2)             :  &
            PRINT CHR (8); " ";CHR(8);       :  &
            GOTO        gs001
   IF       (inchr < 32)  OR  (inchr);
            instr   = instr + CHR(inchr)
            GOTO        gs001
```

To clear the input buffer of type-ahead characters you could use:

```
clear'input:
  XCALL ACCPT,  x,  1
  IF    = 0 RETURN
  GOTO  clear'input
```

CAPTION (AlphaBASIC)

The CAPTION function lets you change the caption for the main terminal emulation window. On a non-GUI terminal, the caption is displayed on the top status line.

Includes

IVTERM.MAP IVTERM.BSI

Input

`cp'title` This is the caption string you want to display. It can be up to 70 characters long.

Output

None.

Example

```
cp'title    ="ABC Construction Inc."
CALL        CAPTION
```

Synonyms

IVTCAP (AMOS)
Change caption line (Direct)

Change caption line (Direct)

This environment function lets you change the caption for the terminal emulation window.

Syntax

`TAB(1,192) T Title` DEL

Title is the string defining the window caption.

Returns

None.

Example

```
MAP1  del    , S, 1, CHR$(127)
PRINT TAB(-1,192);"TJoe's Trucking";del;
```

Synonyms

CAPTION (AlphaBASIC)
IVTCAP (AMOS)

Change cursor shape (Direct)

This environment function changes the shape of the mouse cursor.

Syntax

```
TAB(-1,192) C Shape
```

Shape is a single character that defines the new cursor shape. It may be:

`space`	Default		`!`	Arrow cursor	
`"`	Crosshair cursor		`#`	Text I-beam cursor	
`$`	Square with smaller square inside		`%`	North, east, south, west arrows	
`&`	Northeast, southwest arrows		`'`	North, south arrows	
`(`	Northwest, southeast arrows		`)`	West, east arrows	
`*`	Vertical arrow		`+`	Hour glass	

Returns

None.

Example

```
PRINT TAB(-1,192);"C+"; ! Hour glass
CALL PROCESS'DATA
PRINT TAB(-1,192);"C ";! Default cursor
```

CLEAR'BITMAPS (AlphaBASIC)

This image display function clears the existing bitmaps that the user selected through the SHOW'BITMAP'FROM'LIST call.

Includes

IVTERM.MAP	IVTERM.BSI
IVTBIT.MAP	IVTBIT.BSI

Input

`BL'ID(x)`	ID value of the displayed images (0 if not displayed). This variable is automatically set by the SHOW'BITMAP'FROM'LIST call.
`BL'COUNT`	Number of images in the array.

Output

`BL'X(x)` `BL'Y(x)`	Last known position of image, if it was displayed. Position is in world coordinates (1-10000).

Example

```
CALL        CLEAR'BITMAPS
```

See Also

SHOW'BITMAP'FROM'LIST (AlphaBASIC)

CLEAR'SCREEN (AlphaBASIC)

This environment routine is included as an example of creating a consistent look and feel throughout a program. Used in place of PRINT TAB(-1,0), it clears the screen, puts up a standard title bar, and displays programmed user information. You can modify background and foreground colors to match desired tastes.

This routine assumes the top two lines of the screen are always available for displaying the title.

Includes

IVTERM.MAP IVTERM.BSI

Input

`COMPANY`	A string containing the company or application name.
`TITLE`	A string containing the screen title. This is typically the current process description, for example: "Customer File Maintenance".
`scrfgc`	The main screen foreground color.
`scrgbc`	The main screen background color.
`bndfgc`	The splash band foreground color.
`bndbgc`	The splash band foreground color.

Comments

All the colors are defined in the IVTERM.MAP file. We recommend you make any changes directly into the MAP file. This lets you change colors globally by modifying only one file.

Colors are selected from the standard 16 color AMOS palette.

Output

None.

Example

```
COMPANY     ="ABC Construction Inc.
TITLE       ="Customer File Maintenance"
CALL        CLEAR'SCREEN
```

Close a file (Direct)

This function closes an open file.

Syntax

`TAB(-1,192) GFC Handle` DEL DEL

Handle is the handle returned from the call that opened the file.

Returns

None.

Example

```
MAP1        del     , S   , 1     , CHR$(127)
PRINT       TAB(-1,192);"GFC";STR(HANDLE);del;del;
```

Synonym

CLOSE'FILE (AlphaBASIC)

See Also

Initiate action against a file (Direct)
Read or reread a record (Direct)
Write or rewrite a record (Direct)

CLOSE'FILE (AlphaBASIC)

This function closes an open file. You must call this function for any file you open for reading or writing.

Includes

IVTERM.MAP	IVTERM.BSI
IVTFIL.MAP	IVTFIL.BSI

Input

`FI'ID` The file ID returned from OPEN'FILE.

Output

None.

Example

```
CALL        CLOSE'FILE
```

Synonym

Close a file (Direct)

See Also

OPEN'FILE (AlphaBASIC)
READ'FILE (AlphaBASIC)
WRITE'FILE (AlphaBASIC)

Display a list box (Direct)

This user interface function displays a scrolling Windows list box, from which the user can make a selection.

Syntax

`TAB(-1,192) L Title` [DEL] `Prompt` [DEL] `{x Item` [DEL]`}` [DEL]

Title is the string defining the window caption.

Prompt is the string defining the prompt within the window. This is optional; you can leave it null.

The following data pair can be duplicated up to 1000 times:

> ***x*** is a single character flag to define if this item is the default. An ! sets the item to be the default.
>
> ***Item*** is this item's text.

Returns

`WhcItem` [RETURN]

WhcItem is a string containing the numeric value of the item selected. This value depends on the sequence the items were added to the list. A 0 means the user pressed *Cancel*.

Example

```
MAP1 del   , S, 1, CHR$(127)
MAP1 whcitm, S, 10
PRINT TAB(-1,192);"LPrint Option";del;del;&
" Screen";del;"!Hardcopy";del;" Fax";del;del;
INPUT LINE whcitm
select= VAL(whcitm)
IF select = 0&
      RETURN
ON select GOTO&
      SRCPRT, &
      HRDPRT, &
      FAXPRT
```

Synonym

LIST (AlphaBASIC)

Display a menu (Direct)

This function displays a window containing a button for each menu item passed to it by the program, plus an *Exit* button.

Syntax

`TAB(-1,192) M Title` [DEL] `{x Item` [DEL]`}` [DEL]

Title is the window caption.

The following data pair defines each menu item. It can be duplicated up to 40 times:

x is a space if this menu item is active or `!` to set the item to inactive.

Item is this item's text.

Returns

`WhcItem` [RETURN]

WhcItem is a single ASCII character. Subtract 32 from its ASCII value to determine the menu entry selected. 0 means the user pressed *Exit*.

Example

```
MAP1 del    , S, 1, CHR$(127)
MAP1 whcitm , S, 1
PRINT TAB(-1,192);"MMain Menu";del; &
" Accounts Payable";del;            &
" Accounts Receivable";del;         &
"!Payroll";del;del;

INPUT LINE whcitm
select= ASC(whcitm)-32
IF select = 0&
      RETURN

ON select GOTO  &
      APMENU,         &
      ARMENU,         &
      PRMENU
```

Synonym

MENU (AlphaBASIC)

Display a message box (Direct)

This function displays a standard Windows message box. You set the caption, buttons shown, and message text. You can include multiple lines of text in the message.

Syntax

`TAB(-1,192) B ButnA Icon Dfbt Text` [DEL] `Title` [DEL]

ButnA is one of these characters, defining the buttons to show in the message box:

space	OK	!	OK, Cancel
"	Abort, Retry, Ignore	#	Yes, No, Cancel
$	Yes, No	%	Retry, Cancel

Icon is one of these characters, defining the icon to display:

space	None	!	Stop
"	Question	#	Exclamation
$	Information	A	Stop and audible beep
B	Question and beep	C	Exclamation and audible beep
D	Information and beep		

Dfbt is a single character defining the default button:

space	Button 1	!	Button 2
"	Button 3		

Text is the message. It can be up to 2000 characters long and may include carriage returns (ASCII 13). To force a line break, insert a CHR(13) into the string.

Title is the window caption.

Returns

`Button` [RETURN]

Button is a single ASCII character defining the button the user selected.

Example

```
MAP1 del   , S, 1, CHR$(127)
MAP1 button, S, 1
PRINT TAB(-1,192);"B!#!Are you sure you wish to continue.";CHR$(13); &
      " This operation will delete data.";del;"Purge History";del;
INPUT LINE button
IF button # "!"&
      RETURN
```

Synonym

MESSAGE'BOX (AlphaBASIC)

Display an edit text box (Direct)

This function displays a Windows edit box, in which the user can make or change an entry.

Syntax

`TAB(-1,192) E Title` DEL `Prompt` DEL `Text` DEL

Title is the window caption.

Prompt is the prompt within the window. This is optional; you can leave it null.

Text is the default text to edit. The length of this string defines the user's maximum entry length. Fill this string with spaces to meet the maximum entry length.

Returns

`Text` RETURN

Text is the user's returned string. If the user clicks on Cancel the default text is returned.

Example

```
MAP1 del     S, 1, CHR$(127)
MAP1 usrnam, S, 15
PRINT TAB(-1,192);"EUser Log In";del;"Enter User Name"; &
del;"                ";del;

INPUT LINE usrnam
```

Synonym

EDIT (AlphaBASIC)

Display an extended edit text box (Direct)

This function displays a Windows edit box, in which the user can make or change an entry. It gives you more control over the location of the box and the maximum length of the entry than the plain edit box function.

Syntax

`TAB(-1,192) Ge Title` [DEL] `Prompt` [DEL] `Text` [DEL] `len` [DEL] `x` [DEL] `y` [DEL] [DEL]

Title is the window caption.

Prompt is the window prompt. This is optional; you can leave it null.

Text is the default text to edit.

len is the maximum length of the user's entry.

x is the horizontal position of the left side of the edit box, in pixels.

y is the vertical position of the top of the edit box, in pixels.

Returns

`Text` [RETURN]

Text is the user's entry. If the user selects *Cancel* the default text is returned.

Example

```
MAP1 del   , S, 1, CHR$(127)
MAP1 usrnam, S, 15
PRINT TAB(-1,192);"GeUser Log In";del;"Enter User Name"; &
del;del;"15";del;"50";del;"100";del;del;

INPUT LINE usrnam
```

Synonyms

EDIT (AlphaBASIC)
XEDIT (AlphaBASIC)

EDIT & XEDIT (AlphaBASIC)

The EDIT and XEDIT subroutines displays a Windows edit box in which the user can make an entry. On a non-GUI terminal, they display a simulated edit box at a location you choose (see E'X and E'Y variables, below.)

These routines call the INPUT routine when used on a non-GUI terminal. Therefore, there are these differences on non-GUI terminals:

- The default string is not displayed; instead periods are displayed for each character.
- You cannot enter a string longer than the default string.
- LEN(E'TEXT) always returns 100. INPUT always pads the string to the maximum length with spaces.
- Placement of the edit box is important. If it is too wide to fit on the screen in the location you select, it may corrupt your screen display.

Includes

IVTERM.MAP IVTERM.BSI

Input

`E'TITLE`	A string of up to 50 characters containing the title for the edit box.
`E'PROMPT`	A string of up to 50 characters containing the prompt for the edit box.
`E'TEXT`	A string of up to 100 characters. Load this with the default entry for the edit box. EDIT returns the user's entry into the variable, padded with spaces to the length of the default string. If the user selects *Cancel* or closes the box, the default string is returned.
`E'X(row)` `E'Y(column)`	AMOS floating point variables defining the upper left corner of the simulated edit box when EDIT is used on a non-GUI terminal. Their default values are 12 and 25. On XEDIT these variables control placement of the Windows dialog box on the screen.
`E'LEN`	Available for XEDIT only, the maximum length of the string for data entry. If this value is negative the input string is echoed as * for password security.

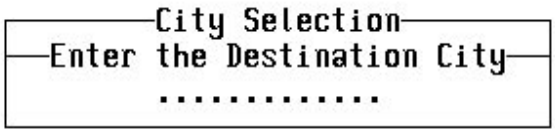
Output

`E'TEXT`	The returned string.
`ABORT`	Available for XEDIT only, is set to TRUE (1) if the user presses *Cancel*.

Example

```
E'TITLE     = "City Selection"
E'PROMPT    = "Enter the Destination City
E'TEXT      = "San Francisco"
E'X         = 10
E'Y         = 25
CALL        EDIT
```

Here is a sample edit box, as displayed on an *in*Sight workstation and a non-*in*Sight terminal:



Synonyms

Display an edit text box (Direct)
Display an extended edit text box (Direct)

Exit Terminal Emulation Window (Direct)

This setup function exits from the terminal emulator running on the workstation. This closes the terminal emulation window as if the user had selected Exit from the pull down menu. When inserted into the final command file of the AMOS application, it gives the effect of shutting down the entire Windows application.

Using this command provides an even higher level of integration between your AMOS-based application and the Windows environment.

Syntax

```
TAB(-1,192) x
```

Returns

None.

Example

As the last operation of an AMOS application you may want to shut down the emulation window. Within your AlphaBASIC program, you could include the following command to do so:

```
PRINT TAB(-1,192); "x";
```

Synonyms

IVTERM'CLOSE (AlphaBASIC)
IEXIT (AMOS)

Free a DLL (Direct)

This function frees a previously loaded DLL.

Syntax

`TAB(-1,192) GdF Handle` DEL DEL

Handle is the handle returned from the call that loaded the DLL.

Returns

None.

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"GdF";STR(HANDLE);del;del;
```

See Also

Load a DLL (Direct)
Make a call into a DLL (Direct)

FSTRIP.SBR (XCALL)

This AlphaBASIC subroutine strips spaces and non-printable characters from both the beginning and end of a string. It is the same as STRIP.SBR except that it also strips leading spaces.

FSTRIP.SBR works with all versions of AlphaBASIC. It is reentrant and reusable and may be loaded into either system or user memory.

Syntax

```
XCALL       FSTRIP,string
```

string is the string to strip spaces from. It must be an AlphaBASIC string type variable. It cannot be a literal string.

Returns

The original string, stripped of leading and trailing spaces.

Example

```
MAP1   cusnam   , S   , 25

cusnam = “   ABC Corp  “
XCALL  FSTRIP, cusnam
```

Errors

If you give FSTRIP the wrong number or type of argument, it returns to the calling program without performing its function.

Get a filename (Direct)

This function prompts the user to select a local file on the workstation. This file is selected using the native file dialog box of the workstation. This call is useful when trying to read files from the workstation or when associating bitmap images with an AMOS application or database.

Syntax

```
TAB(-1,192) GFG Flag Caption [DEL] Filename [DEL]
{{FilDesc|FilWild|}FilDesc|FilWild|...} [DEL] [DEL]
```

Flag defines the dialog box is an “open” or “save as” box. Output a space (ASCII 32) for open, or ! (ASCII 33) for save as dialog box.

Caption is a character string defining the caption of the file input dialog box.

Filename is a character string defining the default path and filespec.

FilDesc is a string defining a filter description. The description shows the user the type of files being selected by the associated FilWild definition. For example, the description for TIFF formatted bitmap images may be "TIFF images (*.TIF)" with a wild card of "*.TIF". The FilDesc/FilWild combination can be duplicated any number of times.

FilWild is a character string defining the wildcard search criteria for the files matching the associated FilDesc.

Returns

```
Filename[RETURN]
```

Filename is a character string containing the newly selected path and filespec. A null string is returned if the user selects *Cancel*.

Example

```
MAP1        del        , S   , 1   , CHR$(127)
MAP1        FILNAM     , S   , 40

PRINT       PRINT TAB(-1,192);"GFG Select an image";del;&
            "C:\IMAGES";del;                            &
            "TIFF images|*.TIF|PCX images|*.pcx";del;del;

INPUT       LINE FILNAM
```

Synonym

GET'FILENAME (AlphaBASIC)

GET'FILENAME (AlphaBASIC)

This function gets a filename from the user. It displays a Windows dialog box for the user to select a file. The file is selected from the native workstation's file system. The selection can be controlled and defaulted to certain paths and filenames.

This routine is useful for selecting an image to associate with a data record. The user can be prompted to select an existing image on the local workstation. Once selected, the file name is returned to AMOS and can then be stored with the data record for later display by the bitmap routines.

Includes

IVTERM.MAP	IVTERM.BSI
IVTFIL.MAP	IVTFIL.BSI

Inputs

`FI'FILNAM`	Default path and filename.
`FI'TITLE`	Title of the window.
`FI'FILCNT`	Number of filters defined.
`FF'DESC(x)`	Description of filters. See below.
`FF'WLDCRD(x)`	File name wildcard to match filter description. See below.
`FI'FLAG`	Flag to determine open file or save as file name. Set it to FF'OPEN to do open dialog box or FF'SAVEAS to set save as dialog box.

Filters

Filters are ways to define groups of files (typically using the file extension). For example, if an application lets the user select from a list of bitmap images the filters could be:

```
FI'FILCNT        = 3
FF'DESC(1)       = "TIFF images (*.TIF)"
FF'WLDCRD(1)     = "*.TIF"
FF'DESC(2)       = "PCX images (*.PCX)"
FF'WLDCRD(2)     = "*.PCX"
FF'DESC(3)       = "BMP images (*.BMP)"
FF'WLDCRD(3)     = "*.BMP"
```

Outputs

`FI'RESULT`	0 if OK, or error. See the error list below.
`FI'FILNAM`	Newly selected filename, "" if user cancels.

Example

```
FI'FILNAM          = "C:\WINDOWS\MARBLE.BMP"
FI'TITLE           = "Select an Image"
FI'FILCNT          = 3
FF'DESC(1)         = "TIFF images (*.TIF)"
FF'WLDCRD(1)       = "*.TIF"
FF'DESC(2)         = "PCX images (*.PCX)"
FF'WLDCRD(2)       = "*.PCX"
FF'DESC(3)         = "BMP images (*.BMP)"
FF'WLDCRD(3)       = "*.BMP"
FI'FLAG            =0
CALL               GET'FILENAME
IF                 FI'FILNAM   = ""&
                   RETURN

BI'FLAGS           = BF'SIZABL
BI'X               =0
BI'Y               =0
BI'FILNAM          = FI'FILNAM
BI'TITLE           = "Selected Image"
CALL               SHOW'BITMAP
```

Errors

Error codes returned in FI'RESULT:

00	OK.	01	File not found.
02	Bad handle.	03	Illegal operation.
04	EOF.	05	Not an *in*Sight workstation.
06	Device error during write.		

Synonym

Get a file name (Direct)

IEXIT (AMOS)

This setup function exits from the terminal emulator running on the workstation. This closes the terminal emulation window on the workstation as if the user had selected Exit from the pull down menu. When inserted into the final command file of the AMOS application, it gives the effect of shutting down the entire Windows application.

Using this command provides an even higher level of integration between your AMOS-based application and the Windows environment. Using IEXIT on a non-*in*Sight-based workstation has no effect. This command can be executed when the workstation is logged off.

Syntax

```
iexit
```

Examples

As the last operation of an AMOS application, you may want to log off the user and shut down the emulation window. Include the following commands in a .CMD file on AMOS that is executed when the user exits from the main application:

```
LOGOFF
iexit
```

Synonyms

IVTERM'CLOSE (AlphaBASIC)
Exit (Direct)

Initiate action against a file (Direct)

This file I/O function sets up access to a specific file. You can open the file for reading, writing or append, look it up or delete it. This call returns the file handle for future access.

You can only read and write straight text files that use CR/LF as record terminators.

Syntax

```
TAB(-1,192) GFA Action Filename [DEL] [DEL]
```

Action is a single character defining the file action to take. It may be:

W	Open the file for writing. The file is deleted if it currently exists.
A	Open the file for append. The file is created if it doesn't exist.
R	Open the file for reading.
D	Delete the file.
L	Look up a file.

Filename is the native filespec of the file you want to access. This may include an entire path and file specification. For example: c:\winorbit\winorbit.ini.

Returns

For W, A and R access:

```
Handle [RETURN]
```

Handle is the file handle. This handle must be used during any future access to the file. This is the way to uniquely identify this file. This value is 0 if the file could not be found or created.

For D and L access:

```
Size [RETURN]
```

Size is the file size in bytes. If not found, this value is 0.

Example

```
MAP1        del      , S  , 1  , CHR$(127)
MAP1        filename , S  , 30 , "C:\WINORBIT\WINORBIT.INI"

PRINT       TAB(-1,192);"GFAL";filename;del;del;
INPUT       SIZE
IF          SIZE = 0                                  &
            RETURN

PRINT       TAB(-1,192);"GFAR";filename;del;del;
INPUT       HANDLE
```

Synonym

OPEN'FILE (AlphaBASIC)

See Also

Close a file (Direct)
Read or reread a record (Direct)
Write or rewrite a record (Direct)

IVT'INFO (AlphaBASIC)

This environment function retrieves information about the workstation display.

Includes

IVTERM.MAP IVTERM.BSI

Input

None.

Output

`IVT'SCRX`	Width, in pixels, of the workstation display.
`IVT'SCRY`	Height, in pixels, of the workstation display.
`IVT'WOFX`	Terminal emulation window offset from the left side of the screen, in pixels.
`IVT'WOFY`	Terminal emulation window offset from the top of the screen, in pixels.
`IVT'WX`	Width, in pixels, of the emulation window.
`IVT'WY`	Height, in pixels, of the emulation window.
`IVT'COLORS`	Total displayable colors on the workstation.

Example

```
CALL        IVT'INFO
PRINT       "Workstation dimensions (" ;      &
            STR(IVT'SCRX);",";STR(IVT'SCRY);")"
```

Synonym

Retrieve workstation information (Direct)

IVT'LAUNCH (AlphaBASIC)

This function launches a program that resides on the workstation. It can include optional parameters the program needs for execution. You may also define the style in which the program gets launched (maximized, minimized, active, not active, etc.).

Includes

IVTERM.MAP IVTERM.BSI

Input

`IL'FLAG` The execution style flag. It may be:

0	Normal execution.
1	Normal (not active).
2	Minimized.
3	Minimized (not active).
4	Maximized.

`IL'COMMAND` The command line for executing the workstation program. This should include the full path, if necessary, and any command line parameters the application needs.

Output

`IL'RESULT` The resulting status code. It may be:

-1	OK.
0	System out of memory.
1	File not found.
2	Path not found.
3 - 21	Windows system error. Refer to the WinExec command for complete details.
100	Not an inSight workstation.

Example

```
IL'FLAG     = 4
IL'COMMAND  "PBRUSH CASTLE.BMP"
CALL        IVT'LAUNCH
```

Errors

See the output section.

Synonym

Launch a workstation program (Direct)

IVTCAP (AMOS)

This environment function changes the caption of the terminal emulation window to any user-selected message. You could set the caption to the company name, application name, module, etc.

On non-*in*Sight terminals the top status line (if any) is set to the message.

Syntax

`ivtcap` *{message}*

message is the message to place in the window title or top status line. If this is blank, the title is cleared.

Examples

To set the title to "ABC Company" you would enter:

```
ivtcap ABC Company
```

Synonyms

CAPTION (AlphaBASIC)
Change Caption Line (Direct)

IVTERM'CLOSE (AlphaBASIC)

This function closes the terminal emulation window. Using it provides an even higher level of integration between your AMOS application and the Windows environment, especially in conjunction with the terminal emulator's auto startup feature. When the user clicks on the Windows icon, the host automatically starts the requested application. When the user is finished, the terminal emulation completely shuts down, giving the illusion of a true Windows program.

Using IVTERM'CLOSE on a non-*in*Sight-based workstation has no effect.

Syntax

```
CALL        IVTERM'CLOSE
```

Returns

None.

Examples

As the last operation of an AMOS application you may want to shut down the emulator window. In your AlphaBASIC program, you could include the following commands:

```
CLOSE       #CUSMAS'CHANNL
CLOSE       #ARMAS'CHANNL
PRINT       TAB(23,1);
CALL        IVTERM'CLOSE
END
```

Synonyms

IEXIT (AMOS)
Exit (Direct)

See Also

IVTERM'INIT (AlphaBASIC)

IVTERM'INIT (AlphaBASIC)

You must call this setup routine before making any other *in*Sight calls in your program. IVTERM'INIT calls NOECHO to set up proper character input for any subsequent *in*Sight calls and sets the following variables which can then be read (but not changed) by the application:

`ivt`	Set to 1 if this is an *in*Sight workstation, to 0 if not.
`pushpop`	Set to 1 if terminal supports save and restore screen areas, to 0 if not.
`multi`	Set to 1 if job is running under MULTI, to 0 if not.
`jobdat`	This structure is filled by JOBDAT.SBR.

Includes

IVTERM.MAP IVTERM.BSI

Syntax

```
CALL  IVTERM'INIT
```

Input

None.

Output

None.

See Also

IVTERM'CLOSE (AlphaBASIC)
JOBDAT.SBR (XCALL)

IVTVER (AMOS)

IVTVER displays the current *in*Sight version on the workstation. On non-*in*Sight terminals, it has no effect.

Syntax

```
IVTVER
```

JOBDAT.SBR (XCALL)

This routine is an AlphaBASIC subroutine designed to return information about your job. You can use this information to control your application or to inform the user of version numbers, etc. This subroutine returns:

- The AMOS system name.
- The attached job name.
- The logical job number.
- The current login.
- The system user name.
- The current AlphaBASIC program name.
- The AlphaBASIC program version.
- Number of rows and columns currently displayed on the terminal.
- Current foreground and background color.
- Terminal flags (has status line, box drawing, *in*Sight capabilities, etc.).

This function works on all versions of AlphaBASIC. It is reentrant and reusable and may be loaded into either system or user memory.

Syntax

```
MAP1   jobdat
 MAP2   jd'sysnam    , S  , 16 !System name
 MAP2   jd'jobnam    , S  , 6  !Job name
 MAP2   jd'jobnum    , F  , 6  !Logical job number
 MAP2   jd'joblog    , S  , 20 !Current login
 MAP2   jd'usrnam    , S  , 20 !User name
 MAP2   jd'prgnam    , S  , 6  !Current program
 MAP2   jd'prgver    , S  , 16 !Program version
 MAP2   jd'trmrow    , F  , 6  !Rows on terminal
 MAP2   jd'trmcol    , F  , 6  !Columns on terminal
 MAP2   jd'trmfgc    , F  , 6  !Foreground color
 MAP2   jd'trmbgc    , F  , 6  !Background color
 MAP2   jd'trmflg(32), B  , 1  !Terminal flags

XCALL   JOBDAT, jobdat
```

jd'trmflg(x) is set to 1 if the terminal has the specific characteristic, or 0 if the characteristic is unavailable. Refer to the *AMOS Monitor Calls Manual* under the TRMCHR command for a complete list of the terminal flags. Here is a quick list:

jd'trmflg(1)	Has alternate page
jd'trmflg(2)	Has block fill
jd'trmflg(3)	Has column insert/delete
jd'trmflg(4)	Has color capability
jd'trmflg(5)	Has multi-key translation
jd'trmflg(6)	Is an Alpha terminal
jd'trmflg(7)	Has status line
jd'trmflg(8)	Has split screen
jd'trmflg(9)	Has erase to end of line
jd'trmflg(10)	Has erase to end of screen
jd'trmflg(11)	Has underscore
jd'trmflg(12)	Has blink
jd'trmflg(13)	Has dim
jd'trmflg(14)	Has reverse video
jd'trmflg(15)	Has character insert/delete
jd'trmflg(16)	Has line insert/delete
jd'trmflg(17)	Has printer support
jd'trmflg(18)	Has 80/132 column support
jd'trmflg(19)	Has smooth scroll
jd'trmflg(20)	Has "box" commands
jd'trmflg(21)	Has "mode" attributes
jd'trmflg(22)	Has AM-70 color commands
jd'trmflg(23)	Has "no space" attributes
jd'trmflg(24)	Has true graphics capability
jd'trmflg(25)	Has grayscale capability
jd'trmflg(26)	Has proportional spacing
jd'trmflg(27)	Has variable row count
jd'trmflg(28)	Has 8-bit character support
jd'trmflg(29)	Has *in*Sight capability

Examples

After setting up the MAP statements as defined above you could display the version of the current AlphaBASIC program in the upper right corner by using the following commands:

```
XCALL   JOBDAT,  jobdat
yyy     = jd'trmcol - LEN(jd'prgver)
PRINT   TAB(1, yyy); jd'prgver;
```

Errors

?Bad argument passed to JOBDAT.SBR.

An incorrect number or type of variables were passed on the XCALL line.

See Also

IVTERM'INIT (AlphaBASIC)

Launch a workstation program (Direct)

This program execution function launches an application that resides on the workstation.

Syntax

`TAB(-1,192) X flag command` [DEL]

Flag determines how the application is launched. It is one of these ASCII characters:

space	Normal execution	!	Normal (not active)
“	Minimized	#	Minimized (not active)
$	Maximized		

Command is the entire application launch command string, including optional parameters, etc.

Returns

If all is OK: `OK` [RETURN]

If there is an error: `error` [RETURN]

error is a numeric string defining the error. See the WinExec command for specifics. Common results are: 0 System out of memory; 1 File not found; 2 Path not found.

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"X$PBRUSH CASTLE.BMP";del;
INPUT LINE A$
IF A$ = "OK"   &
RETURN
ERRCOD= VAL(A$)
```

Synonym

IVT'LAUNCH (AlphaBASIC)

Launch Windows Help System (Direct)

This routine launches the standard Windows help system on the workstation. The topic to search for is also given to provide direct access to a specific section of the help file.

The help file must be created using the Windows help compiler. Refer to a Microsoft Windows help file reference book for details on creating a Windows help file.

Syntax

```
TAB(-1,192) GH helpfile DEL topic DEL DEL
```

helpfile is the Windows help file name. This may include a complete path specification. The default extension is .HLP.

topic is the name of the topic to display.

Returns

None.

Example

```
MAP1 DEL, s, 1 chr$(127)
PRINT TAB(-1,192);"GHC:\WINORBIT\MYHELP";del;"Graphics";del;del;
```

See Also

LAUNCH'HELP (AlphaBASIC)

LAUNCH'HELP (AlphaBASIC)

This function displays one of several help messages, based on the capabilities of the user's terminal or workstation. An associated help file defines the possible help messages. The call determines the best way to display the information without destroying the current contents of the screen. Four display possibilities exist:

- An *in*Sight workstation with no Windows help file displays a Windows message box with a multi-line help message.
- An *in*Sight workstation with a Windows help file is launched into Windows help, directly to the topic of choice.
- A character terminal with push/pop screen displays a multi-line help message in the window on the screen.
- A character terminal with no push/pop screen displays a single help line at the bottom of the screen.

Includes

IVTERM.MAP	IVTERM.BSI
IVTHLP.MAP	IVTHLP.BSI

Input

`HELP'FILNAM` The AMOS file specification of the help file defining the information to present to the user. This file must be formatted as described below.

Comments

The help file defines three types of help information: a single-line message, the Windows help file name and topic, and the multi-line message. It should contain the following:

```
SINGLE=text
```

text is the single line help message.

```
{WINDOWS=winhelp,topic}
```

winhelp defines the Windows help file name and ***topic*** is the topic to search for within the help file. This file must be created using the Windows help compiler. Refer to a Microsoft Windows help file reference book for details on creating a windows help file.

```
{MULTI=title}
text
{/p{title}}
text
\\
```

title defines the title and text of the multi-line help. The title will appear at the top of the help window.

text following the MULTI command is the text of the help message. You may include up to 20 lines of text in this section. End the information by using \\ or end the file. Blank lines are included in the output.

/p, if you include it, causes a page break in the multi-line message. Text following it displays after the break. If you include ***title***, it becomes the new title for the help window.

You can include comment lines in the file by beginning them with a semicolon (;). You may also include blank lines.

There is a sample help file in the Example section, below.

Output

None.

Example

```
HELP'FILNAM = "BAS:CUSNUM.HLP"
CALL        LAUNCH'HELP
```

Here is a sample help file for use with this routine:

```
---BAS:CUSNUM.HLP---

; Help file for the Customer Number
SINGLE=Enter the six character customer number
WINDOWS=C:\AM\AR\ACCTREC.HLP,Customer Number
MULTI=Customer Number

This field defines the customer number associated with an individual
customer’s information.  Enter the number assigned during the creation
of the customer record.

Field type:  Numeric
Max size:  6
Minimum size:  4
Optional:  No
Example:  10200
\\
```

See Also

Launch Windows help system (Direct)

LIST (AlphaBASIC)

The LIST subroutine displays a scrolling Windows list box, from which the user can make a selection. On a non-GUI terminal, it displays a simulated list box at a location you can choose (see L'X and L'Y variables, below.)

Includes

IVTERM.MAP IVTERM.BSI

Input:

`L'TITLE` A string of up to 50 characters containing the title for the list box.

`L'PROMPT` A string of up to 50 characters containing the prompt for the list box.

`L'ITEM'COUNT` An AMOS floating point variable defining the number of items in the list. As defined by Alpha Micro, the list can contain up to 100 items. You can increase this number by changing the size of the L'ITEM array in the IVTERM.MAP file. The absolute maximum size of the array is 1000.

While you can change the number of items allowed in L'ITEM, do not change any other variable sizes. Doing so will keep the subroutine from working correctly. Also, L'ITEM'COUNT must not be larger than the L'ITEM array size.

`L'ITEM` An array with an element for each item on the list. For each element, it contains the following subvariables:

- `L'ITEM'FLAG` Defines whether this item is the default selection. Set it to L'ITEM'DEFAULT for the default item and to L'ITEM'NOTDFLT for all others. If more than one item is set to L'ITEM'DEFAULT, the last one set is the default.
- `L'ITEM'TEXT` Is the text of the item. It can be up to 40 characters.

`L'X(row)`
`L'Y(column)` AMOS floating point variables defining the upper left corner of the simulated list box when LIST is used on a non-GUI terminal. Their default values are 12 and 25.

`L'HIGH` An AMOS floating point variable defining the number of items to show on the screen at a time when LIST is used on a non-GUI terminal. The default is 5.

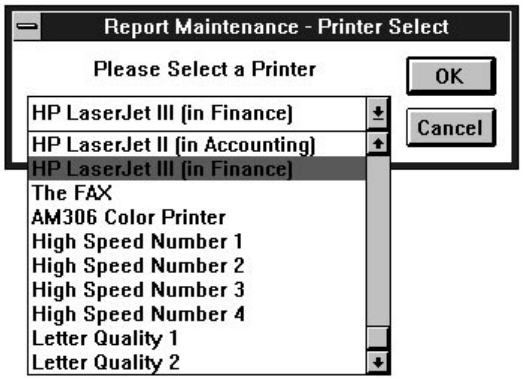
Output

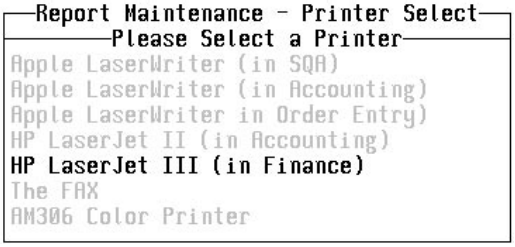
`LIST'RESULT` The item number selected by the user. It contains 0 if the user closes the box or selects CANCEL.

Example

```
L'TITLE          = "State Selection:
L'PROMPT         = "Enter the Destination State"
L'X                = 10
L'Y                = 25
L'HIGH           = 5
L'ITEM'TEXT(1)   = "Alabama"
L'ITEM'FLAG(1)   = L'ITEM'DEFAULT
.
.
L'ITEM'TEXT(50) = "Wyoming"
L'ITEM'FLAG(50) = L'ITEM'NOTDFLT
L'ITEM'COUNT     = 50
CALL             LIST
```

Here is a sample list box, as displayed on an *in*Sight workstation and a non-*in*Sight terminal:





Synonym

Display a list box (Direct)

Load a DLL (Direct)

This function loads a DLL in preparation for calling.

Syntax

`TAB(-1,192) GdL Filename` [DEL] [DEL]

Filename is the PC native file specification of the DLL you wish to load. This may include an entire path and file specification.

Returns

`Handle` [RETURN]

Handle is the DLL handle which uniquely identifies this DLL. You must use this handle during any future access to the DLL. This value is 0 if the DLL could not be found.

Example

```
MAP1 del     , S, 1 , CHR$(127)
MAP1 filename, S, 30, "C:\WINORBIT\TESTDLL.DLL"
PRINT TAB(-1,192);"GdL";filename;del;del;
INPUT HANDLE
IF HANDLE = 0 &
      RETURN
```

See Also

Free a DLL (Direct)
Make a call into a DLL (Direct)

Make a call into a DLL (Direct)

This function calls a function within a previously loaded DLL. You may send data along with the call.

Syntax

`TAB(-1,192) GdC Handle` [DEL] `FunctName` [DEL] `DataLen` [DEL] `Data` [DEL] [DEL]

Handle is the handle returned when the DLL was loaded.

FunctName is the name of the function to call within the DLL.

DataLen is the number of bytes in the ***Data*** area. This must be exact.

Data is the raw data for the function you're calling. This may be in any format as required by the called function.

Returns

None. Nothing is directly returned by this call, although the called function may or may not return data.

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"GdC";STR(HANDLE);del;"TestFunction"; &
del;"10";del;"abcdefghij";del;del;
```

See Also

Free a DLL (Direct)
Load a DLL (Direct)

MENU (AlphaBASIC)

The MENU routine displays a standard Windows menu. The number of each item is the element number of its definition in the M'ITEM array (see below.) MENU returns a floating point value in MENU'RESULT, indicating which menu item the user selected. A zero means *Exit* was chosen; otherwise, the number is the number of the selected menu item.

Windows does not display the item numbers in the menu buttons, only the name of each button from M'ITEM'TEXT. On a non-GUI terminal, MENU displays a plain text menu with the item numbers.

Includes

IVTERM.MAP IVTERM.BSI

Input

`M'TITLE` A string of up to 50 characters containing the title for the menu.

`M'ITEM'COUNT` An AMOS floating point variable defining the number of menu items. The menu can contain a maximum of 40 entries.

`M'ITEM` An array with an element for each item on the menu. For each element, it contains the following subvariables.

- `M'ITEM'FLAG` Defines whether this item is the active or not. If not active the item displays in dim intensity and the user cannot select it. Set M'ITEM'FLAG to M'ITEM'ACTIVE for active entries and to M'ITEM'INACTIVE for inactive entries.
- `M'ITEM'TEXT` The text of the menu item. It can be up to 40 characters long.

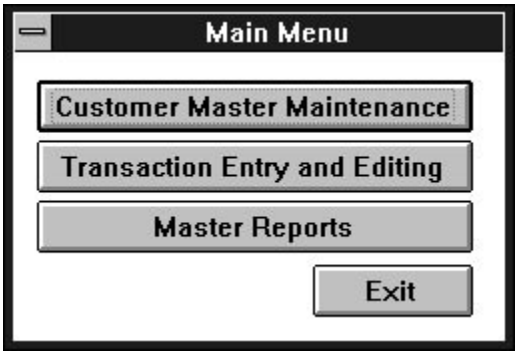
Output

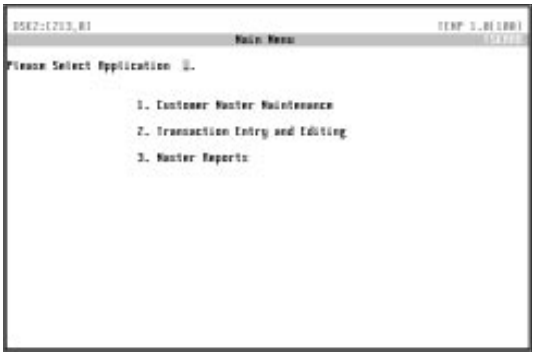
`MENU'RESULT` The menu entry selected by the user. It contains 0 if the user closes the box or selects *Exit*.

Examples

```
M'TITLE          = "Document Inventory"
M'ITEM'TEXT(01)  = "Add New Document"
M'ITEM'FLAG(01)  = M'ITEM'ACTIVE
M'ITEM'TEXT(02)  = "Create Document Inventory Report"
M'ITEM'FLAG(02)  = M'ITEM'ACTIVE
M’ITEM'COUNT     = 2
CALL             MENU
```

Here is a sample menu, as displayed on an *in*Sight workstation and a non-*in*Sight terminal:





Synonym

Display a menu (Direct)

MESSAGE'BOX (AlphaBASIC)

The MESSAGE'BOX subroutine displays a standard Windows message box and returns a code corresponding to the button the user pressed. If the user is not on a GUI workstation, MESSAGE'BOX tries to simulate a message box on the screen or uses the bottom two lines of the screen for input. This call guarantees screen integrity. At most only the last two lines of the screen will be destroyed.

Includes

IVTERM.MAP IVTERM.BSI

Input

`msg'captio` A string of up to 40 characters containing the title for the message box.

`msg'button` The option buttons to be presented to the user for selection. Set this variable to one of the following:

`MB'OK`	OK button only.
`MB'OKCANCEL`	OK and Cancel buttons.
`MB'ABORTRETRYIGNORE`	Abort, Retry and Ignore buttons.
`MB'YESNOCANCEL`	Yes, No and Cancel buttons.
`MB'YESNO`	Yes and No buttons.
`MB'RETRYCANCEL`	Retry and Cancel buttons.

`msg'icon` The icon to display inside the message box. Set this variable to one of the following:

`MB'ICONNONE`	Don't display an icon.
`MB'ICONSTOP`	Stop sign icon.
`MB'ICONQUESTION`	Question mark icon.
`MB'ICONEXCLAMATION`	Exclamation icon.
`MB'ICONINFORMATION`	Information icon.
`MB'ICONSTOPBEEP`	Stop sign with audible beep.
`MB'QUESTIONBEEP`	Question mark with audible beep.
`MB'ICONEXCLAMATIONBEEP`	Exclamation with audible beep.
`MB'ICONINFORMATIONBEEP`	Information with audible beep.

`msg'dftbut` The default button to select. This is the button that will get chosen if the user presses ENTER. Set this variable to one of the following:

`MB'DEFBUTTON1`	First button defined.
`MB'DEFBUTTON2`	Second button defined.
`MB'DEFBUTTON3`	Third button defined.

`msg'text` The text of the message. You can define multiple lines by separating individual lines with a CHR$(13). In a non-GUI environment only the first line is guaranteed to display. The message can be a maximum of 2000 characters, including carriage returns.

Output

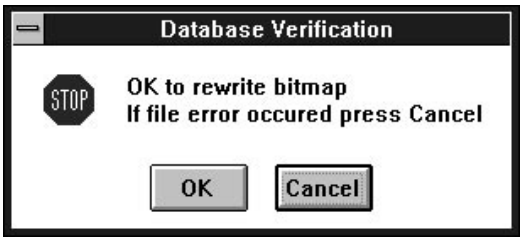
`msg'result` The button selected by the user. It will be equal to one of the following:

`MB'RTNEXIT`	User closed the message box.
`MB'RTNOK`	User pressed OK.
`MB'RTNCANCEL`	User pressed CANCEL.
`MB'RTNABORT`	User pressed ABORT.
`MB'RTNRETRY`	User pressed RETRY.
`MB'RTNIGNORE`	User pressed IGNORE.
`MB'RTNYES`	User pressed YES.
`MB'RTNNO`	User pressed NO.

Example

```
msg'button  = MB'OKCANCEL
msg'icon    = MB'ICONSTOP
msg'dftbut  = MB'DEFBUTTON2
msg'captio  = "Database Verification"
msg'text    = "OK to rewrite bitmap"+CHR$(13) + &
               "If file error occurred press Cancel"

CALL        MESSAGE'BOX
if          msg'result # MB'RTNOK      &
            RETURN
```

Here is a sample message box, as displayed on an *in*Sight workstation and a non-*in*Sight terminal:



Synonym

Display a message box (Direct)

MODIFY'BITMAP (AlphaBASIC)

This image display function changes the characteristics of a bitmap definition.

Includes

IVTERM.MAP	IVTERM.BSI
IVTBIT.MAP	IVTBIT.BSI

Input

`BI'ID` ID value of the displayed image window. This value was returned by the SHOW'BITMAP call.

`BI'ACTION` Desired action to take. This may be:

- `BA'CLEAR` Clear image from screen.
- `BA'FIXPAL` Set palette to match monitor.
- `BA'CONTRA` Set contrast. Value setting may be:
  - `BI'VALUE` 0 - 256, 128 is no change.
- `BA'BRIGHT` Set brightness. Value setting may be:
  - `BI'VALUE` 0 - 256, 128 is no change.
- `BA'ROTATE` Rotate image. Value setting may be:
  - `BI'VALUE` 0 = 90 degrees.
    1 = 180 degrees.
    2 = 270 degrees.
- `BA'CHECK` Check if image is shown.

`BI'VALUE` Value if action requires one, as listed above.

Output

`BI'RESULT` Resulting error code. See the errors section, below.

`BI'VALUE` Resulting value, for these actions only:

- `BA'CLEAR`
- `BA'CHECK`

Return the current (X,Y) coordinate of the image. Result is as follows:

```
Y = INT(BI'VALUE/10000)
X = BI'VALUE-(Y*10000)
```

Example

```
BI'FLAGS    = BF'NOCLR+BF'NOFRAM+BF'FIXPAL+BF'USXY

BI'X        = 0
BI'Y        = 0
BI'FILNAM   = "C:\IMAGES\LOGO.IMG"
BI'TITLE    = " "
CALL        SHOW'BITMAP

XCALL       SLEEP, 5

BI'ACTION   = BA'CLEAR
CALL        MODIFY'BITMAP
```

Errors

Error codes returned in BI'RESULT:

00	OK.	01	No file specified.
02	File not found.	03	Out of memory.
04	Corrupted file.	05	Format not allowed.
06	Bad handle.	07	Can't create file.
08	Page not found.	09	Disk full.

Synonym

Modify bitmap image (Direct)

See Also

SHOW'BITMAP (AlphaBASIC)
SHOW'BITMAP'FROM'LIST (AlphaBASIC)
CLEAR'BITMAPS (AlphaBASIC)

Modify bitmap image (Direct)

This function changes the characteristics of an existing bitmap image window. You can also use it to see if an image is still being displayed and, if so, its current position on the screen.

Syntax

`TAB(-1,192) GcHandle` [DEL] `Action` [DEL] `Value` [DEL] [DEL]

Handle is the handle returned from the show bitmap image function.

Action is a single character string defining the action to take. It may be:

0	Clear image from screen.
1	Set palette to match monitor.
2	Set contrast. Value may be from 0 - 256. 128 is neutral.
3	Set brightness. Value may be from 0 - 256. 128 is neutral.
4	Rotate the image. Value may be: 0 = 90 deg., 1 = 180 deg., or 2 = 270 deg.
5	Check if image is shown.

Value is any value needed by the action, as defined above.

Returns

`Error,Location` [RETURN]

Error is one of these characters, indicating the status of the call:

0	OK	4	Corrupted file.
1	No file specified	5	Format not allowed.
2	File not found.	6	Bad handle.
3	Out of memory.	7	Can’t create file.

Location is the last known position of the image on the screen. The value is returned as a string integer. This integer contains both the X and Y coordinates. The following example code shows how to find each coordinate:

```
loc   =val(location)
x     =int(loc/10000)
y     =mod(loc/10000)
```

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"Gc";STR(HANDLE);del;"0";del;"0";del;del;
INPUT STAT, LOCATION

IF STAT = 0    &
      X= INT(LOCATION/10000): &
      Y = LOCATION - (X*10000)
```

Synonym

MODIFY'BITMAP (AlphaBASIC)

OPEN'FILE (AlphaBASIC)

This file I/O routine opens, deletes or looks up a file. Opening for reading or writing will work only on straight text files where CR/LF is the record terminator.

Includes

IVTERM.MAP	IVTERM.BSI
IVTBIT.MAP	IVTBIT.BSI

Input

`FI'FILNAM` The name of the file, including full path specification.

`FI'ACTION` The action to take against the file. This may be:

`FA'WRITE`	Open file for writing.
`FA'APPEND`	Open file for appending.
`FA'READ`	Open file for reading.
`FA'DELETE`	Delete the file.
`FA'LOOKUP`	Lookup the file.

Output

`FI'RESULT` Result error code. Set to 0 if no error. See error code below.

`FI'ID` ID value of the opened file. This value is required for subsequent access to this particular file.

`FI'SIZE` Delete and lookup return the size in bytes of the file.

Example

```
FI'FILNAM   = "C:\WINDOWS\WIN.INI"
FI'ACTION   = FA'READ
CALL        OPEN'FILE
IF          FI'RESULT # 0    &
      GOTO        NOT FOUND
```

Errors

Error codes returned in FI'RESULT:

00	OK.	01	File not found.
02	Bad handle.	03	Illegal operation.
04	EOF.	05	Not an *in*Sight workstation.
06	Device error during write		

Synonym

Initiate access against a file (Direct)

See Also

CLOSE'FILE (AlphaBASIC)
GET'FILENAME (AlphaBASIC)
READ'FILE (AlphaBASIC)
WRITE'FILE (AlphaBASIC)

PROGRESS (AlphaBASIC)

During a lengthy operation, such as a search through a large data file, it can be a good idea to update the screen display so the user knows the operation is progressing. The PROGRESS routine lets you display and update a progress indicator ("progress bar".) You can use PROGRESS with any operation where you know the end point and can measure how far along the process in. For example, when processing a large file, you can use PROGRESS if you know how many records there are in the file and what record you are currently on.

If your program is scrolling information on the screen, such as account numbers as they are processed, adding a progress bar may corrupt the display of non-GUI terminals. However, with a progress indicator, there is not normally a need to scroll information during processing.

The first time you call PROGRESS for a given operation, it displays the progress bar. Each time you want to update the bar, you should update P'CURRENT and call PROGRESS again. When P'CURRENT equals P'MAX, PROGRESS clears the indicator from the screen.

If you end the operation before it completes, set P'CURRENT equal to P'MAX and call PROGRESS to clear the progress bar from the screen.

You may want to put a routine to clear the progress bar in a Control-C error trap routine to make sure the bar is cleared if someone interrupts the operation.

Includes

IVTERM.MAP IVTERM.BSI

Input

`P'MAX`	A floating point variable containing the total count for this indicator--the 100% mark. It must be a positive integer.
`P'CURRENT`	A floating point variable that is the current position in the process. It must be a positive integer in the range of 0 to P'MAX.
`P'X(row)` `P'Y(column)`	For non-GUI terminals these variables define the upper-left corner of the progress indicator. The default values are 16 and 9.
`P'TITLE`	A string of up to 50 characters containing the title you want for the progress indicator.

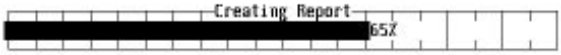
Outputs

None.

Example

```
P'MAX       = RECCNT
P'X         = 10
P'Y         = 9
P'TITLE     = "Report Generation"
FOR         X = 1 TO RECCNT
  CALL      PROCESS'RECORD
  IF        QUIT = 1
            P'CURRENT = P'MAX     :   &
            CALL             PROGRESS  :    &
            RETURN

  P'CURRENT = X
  CALL      PROGRESS
NEXT        X
```

Here is a sample progress indicator, on both an *in*Sight workstation and a non-*in*Sight terminal:



Synonym

Access a progress indicator (Direct)

Read or re-read a record (Direct)

This file I/O routine reads or re-reads a text record from an opened file. Reading reads the next sequential record from the file. Re-reading reads the current record—this is useful if the CRC is incorrect.

You can only read straight text files that use CR/LF as record terminators.

Syntax

`TAB(-1,192) GF Type Handle` [DEL]

Type is the type of read to perform, either `R` to read the next record or `r` to re-read the current record

Handle is the handle returned from the open call

Returns

If the read is successful, the routine returns:

`CRC,Text`[RETURN]

CRC is the calculated CRC of the text of the record. CRC is calculated as follows:

```
MAP1     CRC     ,B     ,2     ,0
FOR      X=1 TO LEN(text)
  CRC    =CRC+(X*ASC(text[X;1]))
NEXT     X
```

Text is the record read from the file.

If the read fails, the routine returns:

`E x`[RETURN]

x is one of these error codes:

0	EOF reached
1	File handle not found
2	File opened with wrong action.

Example

```
        MAP1        del      , S  , 1  , CHR$(127)
        MAP1        INCRC    , B  , 2
        MAP1        CALCCRC  , B  , 2
        MAP1        INLINE   , S  , 500

RN:     PRINT       TAB9-1,1920;"GFR";STR(HANDLE);DEL;DEL
RR:     INPUT       LINE INLINE
        IF          INLINE[1;1] ="E"              &
                    GOTO        RE

        X           = INSTR(1, INLINE, ",")
        INCRC       = VAL(INLINE[1,X-1])
        INLINE      = [INLINE[X+1,-1]
        CALCCRC     = 0

        FOR         X = 1 TO LEN (INLINE)
         CALCCRC    = CALCCRC + (X * ASC(INLINE[X;1])
        NEXT        X
        IF          INCRC # CALCCRC                                &
                    PRINT   TAB(-1,1920;"GFR";STR(HANDLE);DEL;DEL;: &
                    GOTO    RR

        PRINT       INLINE
        GOTO        RN

RE:     FILERR      = VAL(INLINE[2,-1]
        PRINT       "Read error ";FILERR
```

Synonym

READ'FILE (AlphaBASIC)

See Also

Close a file (Direct)
Get a file name (Direct)
Initiate action against a file (Direct)
Write or rewrite a record (Direct)

READ'FILE (AlphaBASIC)

This file I/O routine reads a text line from a file.

For efficiency, load ACCPT.SBR subroutine into user or system memory. This routine is used extensively for the read and write portions of the file I/O functions. Loading it will greatly affect performance.

Includes

IVTERM.MAP	IVTERM.BSI
IVTFIL.MAP	IVTFIL.BSI

Input

`FI'ID`	The file ID returned from OPEN'FILE.

Output

`FI'RESULT`	The result error code. This is set to 0 if there is no error.
`FI'RECORD`	The next record in the file.

Example

```
LOOP:       CALL  READ'FILE
            IF    FI'RESULT # 0     &
                  GOTO           EOF

            PRINT FI'RECORD
            GOTO  LOOP
```

Errors

These error codes may be returned in FI'RESULT:

00	OK.	01	File not found.
02	Bad handle.	03	Illegal operation.
04	EOF.	05	Not an *in*Sight workstation.
06	Device error during write.		

Synonym

Read or re-read a record (Direct)

See Also

CLOSE'FILE (AlphaBASIC)
GET'FILENAME (AlphaBASIC)
OPEN'FILE (AlphaBASIC)
WRITE'FILE (AlphaBASIC)

read'profile'bool (AlphaBASIC)

This routine reads a Boolean value (true/false) from a Windows private profile file.

Includes

IVTERM.MAP IVTERM.BSI
IVTPRO.MAP IVTPRO.BSI

Input

`p'filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, this defaults to *in*Sight's own master profile.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.

Output

`p'val`	The returned value is 0 for false or 1 for true.

Example

```
p'filnam    = "C\ARDATA\CONFIG.INI"
p'section   = "UserInfo"
p'entry     = "HasExcel"
CALL        read'profile'bool
HASEXC      = p'val
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'int (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'int (AlphaBASIC)
write'profile'string (AlphaBASIC)

read'profile'int (AlphaBASIC)

This routine reads an integer value from a private profile file. This is a 16-bit unsigned value.

Includes

IVTERM.MAP	IVTERM.BSI
IVTPRO.MAP	IVTPRO.BSI

Input

`p'filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, this defaults to *in*Sight's own master profile.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.

Output

`p'val`	The returned value from the profile.

Example

```
p'filnam    = "C:\ARDATA\CONFIG.INI"
p'section   = "UserInfo"
p'entry     = "Flag"
CALL        read'profile'int
USRFLG      = p'val
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'bool (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'int (AlphaBASIC)
write'profile'string (AlphaBASIC)

read'profile'string (AlphaBASIC)

This routine reads a string value from a private profile file.

Includes

IVTERM.MAP	IVTERM.BSI
IVTPRO.MAP	IVTPRO.BSI

Input

`p’filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, this defaults to *in*Sight's own master profile.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.

Output

`p'strval`	The returned string from the profile.

Example

```
p'filnam    = "C:\ARDATA\CONFIG.INI"
p'section   = “UserInfo"
p'entry     = "Name"
CALL        read'profile'string
USRSNAM     = p'strval
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'bool (AlphaBASIC)
read'profile'int (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'int (AlphaBASIC)
write'profile'string (AlphaBASIC)

Read or write data from a Private Profile file (Direct)

This function reads data from or writes it to a private profile file located on the PC. The syntax of the file conforms to the standard methods used by Windows.

Syntax

To read a value from a profile:

`TAB(-1,192) GPRType Filename` [DEL] `Section Entry` [DEL] [DEL]

To write a value to a profile:

`TAB(-1,192) GPWType Filename` [DEL] `Section Entry` [DEL] `Value` [DEL] [DEL]

Type is the type of the value. It must be one of the following:

S	For a string	I	For a 16-bit unsigned integer
B	For a Boolean (True/False)		

Filename is the name of the private profile file. This may include an entire path and file specification. If blank, this defaults to the *in*Sight private profile file.

Section is the Profile section heading

Entry is the entry to read or write within the section of the file.

Value is the actual value to store. For strings, include the text. For integers, use the ASCII text representing the value. For Boolean, use 0 (false) or 1 (true).

Returns

When reading: `Value` [RETURN]

Value is the requested information.

When writing: None.

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"GPWIc:\images\image.ini";del;"Configuration";del;&
"Flags";del;"1024";del;del;
```

Synonyms

read'profile'bool (AlphaBASIC)
read'profile'int (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'int (AlphaBASIC)
write'profile'string (AlphaBASIC)

Retrieve workstation information (Direct)

This environment function retrieves information about the current workstation.

Syntax

`TAB(-1,192) GI` [DEL] [DEL]

Returns

`scrx, scry, wofx, wofy, wx, wy, cols` [RETURN]

scrx is the width, in pixels, of display.

scry is the height, in pixels, of display.

wofx is the emulation window left offset.

wofy is the emulation window top offset.

wx is the emulation window width.

wy is the emulation window height.

cols is the total number of displayable colors.

Example

```
MAP1 del, S, 1, CHR$(127)
PRINT TAB(-1,192);"GI";del;del;
INPUT SX, SY, WOX, WOY, WX, WY, COLORS
```

Synonym

IVT'INFO (AlphaBASIC)

SHOW'BITMAP (AlphaBASIC)

This image display function shows a bitmap image. The flags set determine what operations the user may perform on the image.

The user may zoom an image by outlining the zoom area using the mouse. Place the mouse on one corner of the zoom area, press the left button and drag to the opposite zoom corner, then release the button.

Includes

IVTERM.MAP	IVTERM.BSI
IVTBIT.MAP	IVTBIT.BSI

Input

`BI'FLAGS` Combination of the feature flags defining the desired display. Add any combination of the feature flags together and set into BI'FLAGS. Following is a list of the feature flags:

`BF'PRINT`	Allow print. See section on printing, below.
`BF'PLCWIN`	Image placement is emulation window relative; otherwise it is full screen relative.
`BF'NOCLR`	No local clear of image.
`BF'USESCR`	Scroll bars when sizing, else shrink or stretch image based on window size.
`BF'SIZABL`	Sizable border.
`BF'NOTITL`	Don't display captions.
`BF'MENU`	Include user menu.
`BF'NOFRAM`	Display just bitmap.
`BF'FIXPAL`	Do automatic palette conversion to match display
`BF'ROT90`	Rotate 90 degrees.
`BF'ROT180`	Rotate 180 degrees.
`BF'ROT270`	Rotate 270 degrees.
`BF'FLPHOR`	Flip horizontally.
`BF'FLPVER`	Flip vertically.
`BF'ACTIVE`	Leave image window active.
`BF'NOERR`	Don't show errors locally.
`BF'USEXY`	Use application supplied (X,Y) coordinates.
`BF’USEPIX`	Use pixels as coordinate system.

`BI'X`	Vertical position of upper corner of image. Defined in world coordinates in the range of 1-10000 or pixels (BF'USEPIX). Set to 0 to center on the vertical axis.
`BI'Y`	Horizontal position of left corner of image. Defined in world coordinates in the range of 1-10000 or pixels (BF'USEPIX). Set to 0 to center on the vertical axis.
`BI'FILNAM`	File name of bitmap image. This must be a complete DOS file specification, including extension, and may include a complete path.
`BI'XSIZ`	Horizontal size of image. Defined in world coordinates in the range of 1-10000 or pixels (BF'USEPIX).
`BI'YSIZ`	Vertical size of image. Defined in world coordinates in the range of 1-10000 or pixels (BF'USEPIX).
`BI'TITLE`	Window title if being displayed.

Printing

When you set BF'PRINT, the user can print the image by holding down SHIFT and pressing the left mouse button while on the image. This displays a standard printing dialog box.

By default, the image prints in the center of the page, and its height is 50% of the page height. You can change the image size by adding or changing a line in the [IMAGE] section of the INSIGHT.INI private profile file. The line to change is `PrintSize=xx`. ***xx*** is the image height, represented as a percentage of the page height, from 5 to 100.

Output

`BI'RESULT`	Result error code. Set to 0 if no error.
`BI'ID`	ID value of the displayed image. This value is required for subsequent access to this particular image window.

Example

```
BI'FLAGS    = BF'SIZABL+BF'FIXPAL
BI'X        = 0
BI'Y        = 0
BI'FILNAM   = "C:\IMAGES\PHOTO01.BMP"
BI'TITLE    = "Portrait"
CALL        SHOW'BITMAP
```

Errors

These error codes may be returned in BI'RESULT:

00	OK.	01	No file specified.
02	File not found.	03	Out of memory.
04	Corrupted file.	05	Format not allowed.
06	Bad handle.	07	Can’t create file.
08	Page not found.	09	Disk full.

Synonym

Show bitmap image (Direct)

See Also

MODIFY'BITMAP (AlphaBASIC)
SHOW'BITMAP'FROM'LIST (AlphaBASIC)

SHOW'BITMAP'FROM'LIST (AlphaBASIC)

This image display function shows a bitmap image which the user has selected from a list.

Includes

IVTERM.MAP	IVTERM.BSI
IVTBIT.MAP	IVTBIT.BSI

Input

`BL'FLAGS(x)`	Combination of the feature flags defining desired display. These are defined in SHOW'BITMAP.
`BL'X(x)`	Vertical position of upper corner of image, in world coordinates in the range of 1-10000. Set to 0 to center on the vertical axis.
`BL'Y(x)`	Horizontal position of left corner of image, in world coordinates in the range of 1-10000. Set to 0 to center on the horizontal axis.
`BL'FILNAM(x)`	File name of bitmap image. This must be a complete DOS filespec and may include a complete path specification.
`BL'DESC(x)`	Window title (caption) if being displayed.
`BL'COUNT`	Number of images in this array.

Output

`BI'RESULT`	Result error code, as listed below. This is set to 0 if no error.
`BL'RESULT`	Which image element was selected (0 is none or CANCEL).
`BL'ID(BL'RESULT)`	ID value of the displayed image. This value is required for subsequent access to this particular display window.

Example

```
BL'COUNT          = 3
BL'FILNAM(01)     = "c:\images\yvonne.bmp"
BL'DESC(01)       =“portrait”
BL'FLAGS(01)      = BF'USEXY
BL'X(01)          = 1
BL'Y(01)          = 1

BL'FILNAM(02)     = "c:\images\peter.bmp"
BL'DESC(02)       = "Peter"
BL'FLAGS(02)      = BF'USEXY
BL'X(02)          = 8000
BL'Y(02)          = 8000
```

```
BL'FILNAM(03)     = "c:\images\20f.pcx"
BL'DESC(03)       = "Front of Twenty"
BL'FLAGS(03)      = 0
BL'X(03)          = 1
BL'Y(03)          =

LF:    CALL       SHOW'BITMAP'FROM'LIST
       IF         BL'RESULT # 0         &
                  GOTO       LP

       CALL       CLEAR'BITMAP
```

Errors

These error codes may be returned in BI'RESULT:

00	OK.	01	No file specified.
02	File not found.	03	Out of memory.
04	Corrupted file.	05	Format not allowed.
06	Bad handle.	07	Can’t create file.
08	Page not found.	09	Disk full.

See Also

MODIFY'BITMAP (AlphaBASIC)
CLEAR'BITMAPS (AlphaBASIC)
SHOW'BITMAP (AlphaBASIC)

Show bitmap image (Direct)

This function displays a bitmap image on the workstation. Flags are available to control the method of display.

The user may zoom an image by outlining the zoom area using the mouse. Place the mouse on one corner of the zoom area, press the left button and drag to the opposite zoom corner, then release the button.

Syntax

```
TAB(-1,192) GbFlag DEL x{,xsiz} DEL y{,ysiz} DEL File DEL
            Caption DEL DEL
```

Flag is a character string defining the display options. Add the values of the desired flags from this list to get the string you want:

1	Allow print. See Printing section of SHOW'BITMAP reference sheet.
2	Image placement is emulation window relative; else full screen relative.
4	No local clear of image.
8	Scroll bars when sizing; else shrink or stretch image based on window size.
16	Sizable border.
32	Don't display caption.
64	Include user menu.
128	Just bitmap.
256	Do automatic palette conversion to match display.
512	Rotate 90 degrees.
1024	Rotate 180 degrees.
1536	Rotate 270 degrees.
2048	Flip horizontally.
4096	Flip vertically.
8192	Leave image window active.
16384	Don't show errors locally.
32768	Use application supplied (X,Y) coordinates.
65536	Use pixels as coordinate system.

x is a character string, in the range of 1 to 10000, defining the horizontal position of the left edge of the image. 0 centers the image on the horizontal axis. This parameter is evaluated only if 32768 is set in the flag parameter.

xsiz is the horizontal size of the image.

y is a character string, in the range of 1 to 10000, defining the vertical position of the top edge of the image. 0 centers the image on the vertical axis. This parameter is evaluated only if 32768 is set in the flag parameter.

ysiz is the vertical size of the image.

File is the file specification of the image. This may include an entire path and file specification. For example: c:\images\bob.tif.

Caption is the caption of the window if the title bar is being displayed.

Returns

`Error,Handle` RETURN

Error is a character string indicating the status of the display:

0	OK	4	Corrupted file.
1	No file specified	5	Format not allowed.
2	File not found.	6	Bad handle.
3	Out of memory.	7	Can't create file.

Handle is the handle which uniquely identifies this image window. You must use this handle during any calls to the modify bitmap function.

Example

```
MAP1 del, S, 1, CHR$(127)

PRINT TAB (-1,192);"Gb272";del;"0";del;"0";del;  &
"c:\images\bob.tif";del;"Portrait";del;del;

INPUT STAT, HANDLE
```

Synonym

SHOW'BITMAP (AlphaBASIC)

WINFLG.SBR (XCALL)

WINFLG.SBR is an AlphaBASIC subroutine designed to return *in*Sight information about your job. You can then use this information to control your application. In its simplest form WINFLG returns a 1 if *in*Sight is running on the attached workstation or 0 if it isn't.

You can also use WINFLG.SBR to check the other terminal attribute flags and whether a particular TCRT call is valid on the attached terminal. You can also determine if this is a child job to a MULTI or AlphaOFFICE job.

WINFLG.SBR works on all versions of AlphaBASIC. It is reentrant and reusable and may be loaded into either system or user memory.

Syntax

If you just want to see if *in*Sight is running:

```
MAP1   IVT     ,F  ,6
XCALL  WINFLG,ivt
```

ivt is set to 1 if *in*Sight is running or to 0 if unavailable.

To check other terminal and job characteristics:

```
MAP1 trmflg , F  , 6
MAP1 flg    , F  , 6
!    flg  = 0 - 31    Check terminal attribute flags
!    flg  = 32 - 255  Check for valid TCRT
!    flg  = 256       Check for MULTI and OFFICE in parent
XCALL WINFLG, trmflg, flg
```

trmflg is set to 1 if the attribute is available or the TCRT call is valid, 0 if the characteristic is not available.

Comments

These are the terminal attribute positions:

0	Alternate page	15	Line insert/delete
1	Block fill	16	Printer support
2	Column insert/delete	17	80/132 column support
3	Color capability	18	Smooth scroll
4	Multi-key translation	19	“Box" commands
5	Is an Alpha Micro terminal	20	"Mode" attributes
6	Status line	21	AM-70 color commands
7	Split screen	22	"No space" attributes
8	Erase to end of line	23	True graphics capability

9	Erase to end of screen	24	Grayscale capability
10	Underscore	25	Proportional spacing
11	Blink	26	Variable row count
12	Dim	27	8-bit character support
13	Reverse video	28	*in*Sight capability
14	Character insert/delete		

Examples

To determine if the attached terminal is running *in*Sight:

```
XCALL WINFLG, ivt
```

If *in*Sight is available ***ivt*** is set to 1; otherwise it is 0.

To check for other terminal attributes, add a second floating point variable that specifies which attribute to check. To check for 132 column support use the following statement:

```
XCALL WINFLG, wide, 17.0
```

wide is a floating point variable that receives the result. If the attribute is available, it is set to 1; otherwise it is 0.

17.0 is a floating point value telling WINFLG that you want to check the 80/132 column attribute. See the table above for a complete list of attribute positions.

If the second floating point variable is between 32 and 255, WINFLG checks to see if a particular TCRT (save screen, set cursor to blink, etc.) is available For example, to check for save screen area (TCRT code 148), use the following statement:

```
XCALL WINFLG, svar, 148.0
```

If the TCRT is available then ***svar*** is set to 1, otherwise it is 0.

Errors

?Bad argument passed to WINFLG.SBR

An incorrect number or type of variables were passed on the XCALL line.

See Also

IVTERM'INIT (AlphaBASIC)

Write or rewrite a record (Direct)

This file I/O routine writes or rewrites a text record to an opened file. Writing writes to the next sequential record in the file. Rewriting writes over the current record, which can be useful if the CRC is incorrect.

This routine can only write to straight text files. A CR/LF is appended to the written record.

Syntax

`TAB(-1,192) GF Type Handle` [DEL] `Text` [DEL] [DEL]

Type is the type of write to perform, either `W` to write the next record or `w` to rewrite the current record.

Handle is the handle returned from the open call.

Text is the text to write to the record.

Returns

If the write is successful:

`CRC,Text`[RETURN]

CRC is the calculated CRC of the text of the record, calculated as follows:

```
MAP1     CRC     ,B     ,2     ,0
FOR      XY=1 TO LEN(text)
  CRC    =CRC+(X*ASC(text[X;1]))
NEXT X
```

If the write is unsuccessful:

`E x`[RETURN]

x is one of these error codes:

1	File handle not found.
2	File opened with wrong action.
3	File error.

Example

```
        MAP1  del      , S  , 1  , CHR$(127)
        MAP1  INCRC    , B  , 2
        MAP1  CALCCRC  , B  , 2
        MAP1  RTNSTS   , S  , 20
        MAP1  OUTLINE  , S  , 500

        FOR   Y = 1 TO 20
          OUTLINE   = "This is a test "+(Y USING"##")
          CALL      WR
        NEXT  Y
        END

WR:     FILERR      = -1
        PRINT TAB(-1,192);"GFW";STR(HANDLE);del;&
              OUTLINE;del;del;
RW:     INPUT LINE RTNSTS
        IF    RTNSTS[1;1] = "E"
              GOTO        WE

        INCRC = VAL(RTNSTS)
        CALCCRC     = 0

        FOR   X = 1 TO LEN(OUTLINE)
          CALCCRC   = CALCCRC + (X*ASC(OUTLINE[X;1])
        NEXT  X
        IF    INCRC # CALCCRC
              PRINT  (TAB(-1,192);"GFW";STR(HANDLE);del;del;&
              del;OUTLINE;del;del;
              GOTO   RW

              RETURN

WE:     FILERR      = VAL(INLINE[2,-1]
        PRINT "Write error ";FILERR
        RETURN
```

Synonym

WRITE'FILE (AlphaBASIC)

See Also

Close a file (Direct)
Get a file name (Direct)
Initiate action against a file (Direct)
Read or re-read a record (Direct)

WRITE'FILE (AlphaBASIC)

This file I/O function writes a text line into a file.

For efficiency, load the ACCPT.SBR subroutine into user or system memory. This routine is used extensively for the read and write portions of the file I/O functions. Loading it will greatly effect performance.

Includes

IVTERM.MAP	IVTERM.BSI
IVTFIL.MAP	IVTFIL.BSI

Input

`FI'ID`	The file ID returned from OPEN'FILE.
`FI'RECORD`	The record to write to the file.

Output

`FI'RESULT`	The result error code, as listed below. This is 0 if no error.

Example

```
FOR   X = 1 TO 10
        FI'RECORD        = "This is line "+(X USING "##)
        CALL             WRITE'FILE
      NEXT  X
```

Errors

The error code returned in FI'RESULT may be:

00	OK.	01	File not found.
02	Bad handle.	03	Illegal operation.
04	EOF.	05	Not an *in*Sight workstation.
06	Device error during write.		

Synonym

Write or rewrite a record (Direct)

See Also

CLOSE'FILE (AlphaBASIC)
GET'FILENAME (AlphaBASIC)
OPEN'FILE (AlphaBASIC)
READ'FILE (AlphaBASIC)

write'profile'bool (AlphaBASIC)

This routine writes a Boolean value (true or false) to a private profile file.

Includes

IVTERM.MAP IVTERM.BSI
IVTPRO.MAP IVTPRO.BSI

Input

`p'filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, it defaults to *in*Sight's own master profile file.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.
`p'val`	The Boolean value to store. Must be 1 for true or 0 for false.

Output

None.

Example

```
p'filnam    = "C\ARDATA\CONFIG.INI"
p'section   = "UserInfo"
p'entry     = "HasExcel"
p'val       = HASEXC
CALL        write'profile'bool
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'bool (AlphaBASIC)
read'profile'int (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'int (AlphaBASIC)
write'profile'string (AlphaBASIC)

write'profile'int (AlphaBASIC)

This routine writes an integer value to a private profile file. This is a 16-bit unsigned value.

Includes

IVTERM.MAP	IVTERM.BSI
IVTPRO.MAP	IVTPRO.BSI

Input

`p'filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, it defaults to *in*Sight's own master profile file.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.
`p'val`	The integer value to store.

Output

None.

Example

```
p'filnam    = "C\ARDATA\CONFIG.INI"
p'section   = "UserInfo"
p'entry     = "Flag"
p'val       = USRFLG
CALL        write'profile'int
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'bool (AlphaBASIC)
read'profile'int (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'string (AlphaBASIC)

write'profile'string (AlphaBASIC)

This routine writes a string to a private profile file.

Includes

IVTERM.MAP	IVTERM.BSI
IVTHLP.MAP	IVTHLP.BSI

Input

`p'filnam`	The name of the private profile file on the PC. This is a DOS file containing all the profile information. If left blank, it defaults to *in*Sight's own master profile file.
`p'section`	The section within the profile.
`p'entry`	The entry within the profile.
`p'strval`	The string to store in the profile.

Output

None.

Example

```
p'filnam    = "C\ARDATA\CONFIG.INI"
p'section   = "UserInfo"
p'entry     = "Name"
p'strval    = USRNAM
CALL        write'profile'string
```

Synonym

Read or write data in a private profile file (Direct)

See Also

read'profile'bool (AlphaBASIC)
read'profile'int (AlphaBASIC)
read'profile'string (AlphaBASIC)
write'profile'bool (AlphaBASIC)
write'profile'int (AlphaBASIC)

CHAPTER FIVE
THE ALPHAACCOUNTING SUBROUTINES

*in*Sight for AMOS includes four AlphaACCOUNTING subroutines which have been updated to use GUI display features. Since these XCALLs use the subroutines described in Chapter 4, they will display correctly on both GUI workstations and character-oriented terminals.

Many applications, whether rooted in AlphaACCOUNTING or not, use these subroutines. We have provided these updated versions to show how easy it can be to upgrade an existing application to use *in*Sight.

The following pages contains reference sheets for these four XCALLs:

- ANYCN
- MESAG
- MMENU
- TMENU

The way you use these calls under *in*Sight is the same as the way they have always been used; these reference sheets are included as a convenience to those of you who have not previously used the AlphaACCOUNTING subroutines. They are slightly modified versions of the documentation list file which accompanies the AlphaACCOUNTING software.

If your programs already use these XCALLs, they do not need any modification to work with the new *in*Sight versions. When you run a program using one of these routines on a workstation with *in*Sight, it will use the GUI features. These subroutines do not require you to use the include files and subroutines described in Chapter 4.

ANYCN Check for Change

ANYCN asks if the user wants to change any of the data just entered. If so, ANYCN optionally asks for the number of the item to be changed. All ANYCN input and output occurs on line 24 of the terminal screen.

If you use ANYCN in a GUI environment, ANYCN displays its prompts in a standard dialog box.

ANYCN starts by asking `ANY CHANGE?`, to which the user may respond **Y**[RETURN], **N**[RETURN], **END**[RETURN], a backspace (the same as END), or with a decimal number. A decimal number implies Y and also selects the item to change. If the user enters **Y**[RETURN] and your program is not using ANYCN option 2, ANYCN next asks `WHAT #?`. ANYCN then accepts a decimal number selecting the item to change.

Format

```
XCALL ANYCN,cngct1,whatno
```

cngctl is the control variable. The values your program may place in this variable are:

2	Don't ask for number. (That is, don't display the message `WHAT #?`.)
3	Display an error message (`INVALID SELECTION - CR TO RECOVER`), then ask for changes.
4	Ask for number; that is, display `WHAT #?`.
5	Allow 0 to be input for `WHAT #?`.

The values returned to your program in cngctl are:

-1	User typed `END`.
0	User typed `N`.
1	User typed `Y`.

If ANYCN asked `WHAT #?`, ***whatno*** contains the decimal number if the user entered one.

Notes

ANYCN uses INPUT.SBR and MESAG.SBR, both of which must be loaded into system or user memory for ANYCN to run properly.

Your program must call the NOECHO subroutine before using ANYCN.

ANYCN is fully reentrant and may be loaded into system memory.

MESAG Display Standard Error Messages

MESAG outputs standard error messages to the terminal screen. If you use MESAG in a GUI environment, it displays a standard message box.

Format

```
XCALL MESAG,msg,msxctl
```

msg is a string containing the message to display. In options 1,2, and 4, shown below, you may enter any message. In options 3, 5, and 6, the message is predetermined and unchangeable.

msxctl is a numeric filed specifying the message option:

1 Display message and `CR TO RECOVER` on line 24 of the screen
2 Display message and `CR TO CONTINUE` on line 24 of the screen.
3 Display `INVALID ENTRY SEE INSTRUCTIONS` and `CR TO RECOVER` on line 24 of the screen. When this option is used, msg should be null.
4 Display message only on line 24 of the screen.
5 Display `FILE NOW FULL - SEE INSTRUCTIONS` and stop. If this option is used, all files should be closed prior to calling MESAG. msg should be null. The program exits to the monitor.
6 Display `INVALID SELECTION` and `CR TO RECOVER` on line 24 of the screen. When this option is used, msg should be null.

Notes

Your program must call the NOECHO subroutine before using MESAG. MESAG is completely reentrant and may be loaded into system memory.

MMENU Display Standard Master Menu

The MMENU subroutine displays the standard master file maintenance menu and accepts a selection. The name of the application is displayed in the upper left-hand corner of the screen, and this menu is displayed in the middle of the screen:

```
PLEASE SELECT APPLICATION:

     1.  ADD NEW recnam
     2.  CHANGE/INQUIRE recnam
     3.  DELETE recnam
     4.  PRINT-OUT recnam
```

recnam is a string supplied by your program that specifies the item to be maintained (see below).

If the user chooses option 4 (PRINT-OUT), the user is asked if the data should be sorted before printing. The answer to this question is reflected in the value of SELECT returned by MMENU.

If you use MMENU in a GUI environment, it displays its items in a standard menu box.

Format

```
XCALL MMENU,menuid,recnam,select
```

menuid is a string specifying the title of the application. It is displayed in the upper left corner of the screen.

recnam is a string specifying the name of the key field of the master record to be maintained (CUSTOMER, VENDOR, etc.).

select contains the value of the application selected by the user, where:

0	User typed `END`.
1	User selected ADD mode.
2	User selected CHANGE/INQUIRE mode.
3	User selected DELETE mode.
4	User selected PRINT-OUT.
5	User selected PRINT-OUT with sort before printing

Notes

MMENU uses INPUT.SBR, MESAG.SBR, and WAIT.SBR, all of which must be loaded into user or system memory for MMENU to run properly.

Your program must call the NOECHO subroutine before MMENU.

MMENU is totally reentrant and may be loaded into system memory.

TMENU Display Standard Transaction Menu

The TMENU subroutine provides a convenient way of displaying the standardized transaction menu and accepting user response. The standard transaction menu consists of a title supplied by your program on line 1 and the following menu centered on the screen:

```
PLEASE SELECT APPLICATION:

     1.  ENTER (ADD) NEW TRX
     2.  CHANGE EXISTING TRX
     3.  DELETE EXISTING TRX
     4.  PRINT TRX EDIT LIST
     5.  POST TRX TO MAIN FILE
     6.  CREATE BACKUP WORK FILE
     7.  RESTORE BACKUP WORK FILE
```

If you use TMENU in a GUI environment, it displays its items in a standard menu box.

Format

```
XCALL TMENU,menuid,select
```

menuid is a string specifying the title to be displayed on line 1 of the screen.

select contains the user response returned by TMENU:

0	User input END.
1	User selected Add new transaction.
2	User selected Change existing transaction.
3	User selected Delete existing transaction.
4	User selected Print transactions to main file.
5	User selected Create backup work file.
6	User selected Restore backup work file.

Notes

TMENU uses INPUT.SBR, which must be loaded into system or user memory before you call TMENU.SBR.

Your program must call the NOECHO subroutine before TMENU.

TMENU is fully reentrant and may be loaded into system memory.

CHAPTER SIX
PROGRAMMING AND INTEGRATION METHODS

This chapter explains some *in*Sight integration methods. Adding *in*Sight to a number of applications has revealed a few logical approaches to this integration. These steps insert some of the *in*Sight look into every program very quickly, with more and more integration over time. This lets you get your application into Windows today, released and fully functional. And, it will still be fully releasable as you add each new *in*Sight feature. You may only need a small amount of integration, or you may want to take advantage of every aspect of *in*Sight—either way you can add capacity in a gradual and controlled fashion.

PHILOSOPHY

Before getting into the coding details and techniques, let's review *in*Sight's basic philosophy. *in*Sight is intended to bring AMOS legacy applications together with modern tools and methodologies, using the PC workstation running Windows. The existing software applications for Windows are attractive to users; they expect to have all of their applications run in this environment. By using a Windows based terminal emulation package like AlphaLAN 5.0 you can run your AMOS application alongside programs such as Word Perfect. AlphaLAN 5.0 offers connections to the AMOS host in a variety of fashions: standard serial cables, AlphaNET over Ethernet, TCP/IP over Ethernet, and more. So making the connection is easy.

Once the application runs under Windows half the battle is done, and you can start to add the features that will truly show you're a Windows application. The *in*Sight applications tool kit is a collection of GUI functions designed to enhance the look and feel of your application. It includes features such as graphical menus, message and list boxes, progress indicators, and even imaging. All of this while co-existing with character based terminals like your AM-65. In fact, character-based terminals also benefit, as some of the *in*Sight tools provide similar functional improvements for them.

If you need more imaging abilities than *in*Sight provides, the image.doc product provides a complete image management system with simple integration.

THE ALPHALAN ENVIRONMENT

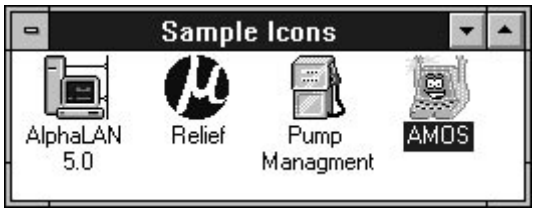
Before making code modifications, there are some simple cosmetic enhancements you can make with AlphaLAN which do little functionally but greatly improve your application's appearance.

- By using a START.CMD file in AlphaLAN's working directory on the workstation, you can automatically launch your application when the user selects AlphaLAN. The

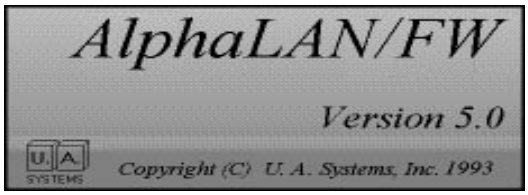
commands in the start file are sent to the host when AlphaLAN is executed. You can have AlphaLAN log the user on and start your application. Here is a sample file:

```
LOG FRED SMITH
PAYROL
```

- Change the icon for the AlphaLAN selection on the desktop. Use any of the thousands of commercially available icons or create your own using a commercial or shareware icon editor. You can even have multiple icons to access different modules of your application. Here are the default AlphaLAN icon and some custom ones:



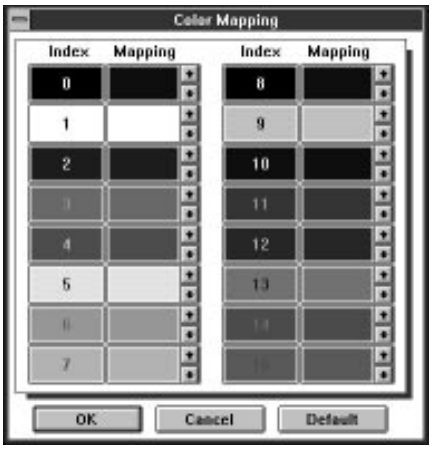
- AlphaLAN 5.0 brings up a bitmap image for a couple of seconds while it's initializing. You can display the bitmap of your choice during this time. Using a tool as simple as Paint Brush or as sophisticated as Photoshop you can create a bitmap that represents your company or application. For best results, it should be between 200 by 100 pixels and 600 by 400 pixels and saved as a .BMP format image. To display your bitmap file, edit the WINORBIT.INI file in your AlphaLAN directory. In the `[userpref]` section find the `bitmap=` line and change the file name to your new logo file. The AlphaLAN default window is shown below-left and a custom bitmap is to the right.



- Add up to 48 user-defined screen buttons which send a character sequence to your application when they are selected. For example, if your application uses up and down arrows to move from field to field you can assign the up arrow (CTRL-K) to a button labeled *Prev Fld* and down arrow (CTRL-J) to a *Next Fld* button. For something like AlphaWRITE you could put the SPELL command (ESC ESC SPELL) behind a button. The button bar below shows a complete set of application specific functions. You define buttons using the User Defined Keys command from the Others Menu in AlphaLAN.

- For quick and dirty colorization you can use the AlphaLAN color mapping option to change one color to another. If the background is normally black (like on an AM-65) you can change black to dark blue, then change green to white to make the text white. This

gives you a more “Windows” look with no code modifications. To get to the color map make these choices from the AlphaLAN Main Menu: Settings, Display Options, Color Mapping. This method is not intended to replace actual colorization of your application, but it does offer a quick way to get the look, giving you time to make the actual color calls within the program code. Additional discussions and personal preferences about colorization are later in this chapter.



MODIFYING YOUR PROGRAM

With the simple configuration items—which can make a dramatic improvement by themselves—out of the way, you’re ready to make some code changes. These changes will blend the GUI features available in the *in*Sight tool kit into your application.

Our examples use AlphaBASIC as the programming language. While you can use *in*Sight from any language by directly outputting the appropriate escape sequences, the main thrust has been in supporting AlphaBASIC. This is done through the use of include files which simplify access to the *in*Sight function library. These include files support, where possible, character terminals. This means most of the functions work regardless of the type of terminal connected. Also, because the character terminal routines are written in AlphaBASIC, you can modify them to meet any specific requirements of your application.

For the rest of this chapter, whenever we say terminal, we mean a character-based terminal such as the AM-65, as opposed to a Windows-based workstation.

Setup

To take advantage of the *in*Sight routines you must include the appropriate support files in your program. At the most basic level you must include IVTERM.MAP toward the top of your program and IVTERM.BSI at the bottom. Prior to calling any *in*Sight functions you must call IVTERM'INIT. This sets up the appropriate work variables and sets No Echo for your job. The variable `ivt` is set to 0 if the attached terminal is not *in*Sight compatible or set to 1 if it is. Keep the `ivt` variable in mind as we will use it later to make some logic decisions.

For other languages (assembly, C, etc.) you should determine if *in*Sight is available on the attached workstation by checking the TD$ENH flag (^O2,000,000,000) in the terminal driver header. You could use the following code to make the determination from assembly:

```
TD$ENH   = ^O2000000000          ; For earlier AMOS releases
GETIMP   TC.SIZ, A5              ; Get TRMCHR memory
TRMCHR   @A5                     ; Get the terminal characteristics
MOV      TC.FLG(A5), D7          ; Get the Flags
AND      #TD$ENH, D7             ; Check for inSight
BNE      GOTINS                  ; inSight is on the attached workstation
```

Within the code samples, all **bolded** items are defined inside the *in*Sight include modules. Any other subroutines or variable definitions are for example purposes only and should match the existing application for similar functionality.

Caption

One of the simplest routines to use is the caption change function. This lets you change the title on the top of the AlphaLAN window to any text you choose. You may want to display the application name, user's company name, current program function, etc. For example, to change the caption to the user's company name, place the following code in your main menu program:

```
CALL                        GET'CONAME  ! Get the company name from
                                        ! application database
C'CAPTION'STRING            = CONAME    ! Set the inSight variable to
                                        !  the company name
CALL                        CAPTION     ! Set the caption
```

Menu

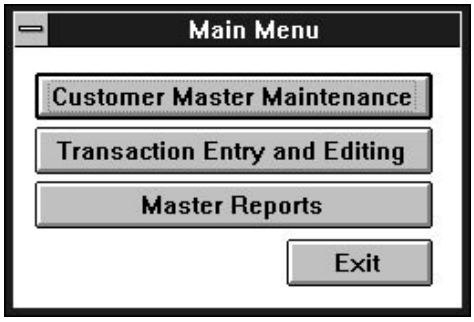
Changing to *in*Sight style menus gives the user point and click access to your various functions. To use this feature you must first decide if you want to give up your existing style of menu on your terminals or keep your existing style and just add the *in*Sight menu if the user is running on a workstation. The menu routine includes simple terminal support which may or may not meet your demands. If not, it is easy to use your existing method and call *in*Sight only when using a workstation.

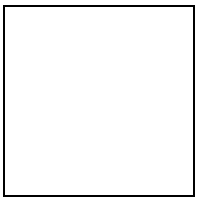
To use the *in*Sight menu for all terminals, follow the example below. This sample code assumes the array PROGRAM(xx) has been predefined with the program .RUN names of the various modules.

```
M'TITLE              = "Main Menu"          ! Set the title
M'ITEM'COUNT         = 3                    ! Define the number
                                            ! of items in this menu
M'ITEM'TEXT(1)       = "Customer Master Maintenance"
M'ITEM'FLAG(1)       = M'ITEM'ACTIVE
```

```
M'ITEM'TEXT(2)       = "Transaction Entry and Editing"
M'ITEM'FLAG(2)       = M'ITEM'ACTIVE
M'ITEM'TEXT(3)       = "Master Reports"
M'ITEM'FLAG(3)       = M'ITEM'ACTIVE
CALL                       MENU                    ! Call the inSight menu
                                                   ! function
IF MENU'RESULT = 0                      &
         GOTO              EXIT
CHAIN                      PROGRAM(MENU'RESULT)
```

Below is output produced by this code. The left shows an *in*Sight workstation; the right is a terminal:





If you don't want to use the *in*Sight menus on a terminal, just bypass the MENU call if not on an *in*Sight workstation. Here is an example:

```
MY'MENU:
   IF ivt = 1              &
         GOTO  insight'menu

! Normal menu code....

GOT'SELECTION:
! Normal chain code

insight'menu:
   M'TITLE           = "Main Menu"     ! Set the title
   M'ITEM'COUNT      = 3               ! Number of items in this menu
   M'ITEM'TEXT(1)    = "Customer Master Maintenance"
   M'ITEM'FLAG(1)    = M'ITEM'ACTIVE
   M'ITEM'TEXT(2)    = "Transaction Entry and Editing"
   M'ITEM'FLAG(2)    = M'ITEM'ACTIVE
   M'ITEM'TEXT(3)    = "Master Reports"
   M'ITEM'FLAG(3)    = M'ITEM'ACTIVE
   CALL              MENU              ! Call the inSight menu function
   MY'SELECTION      = MENU'RESULT
   GOTO              GOT'SELECTION
```

Carriage Return To Continue

We all use routines to put a message on the 24th line and request the user to press carriage return to continue. Yours may be an include function in your common library, an XCALL routine of your own design or the stock XCALL MESAG routine. By modifying your routine you can display a Windows message box instead of the dull 24th line output. With the XCALL you make the change and all of your programs get the advantage automatically. With an include file you'll have to recompile your programs to get the benefit.

Let's examine the various methods. First, and by far the easiest, if you are using the stock XCALL MESAG subroutine replace it with the MESAG.VBR included with *in*Sight. If you are running *in*Sight, this modified XCALL puts up a Windows message box. If not, it uses the old method. To install this new routine, follow the instructions in Chapter 2.

If you use your own XCALL, you need to modify it to check for *in*Sight, then output the escape characters directly. Here is an example of how this might be accomplished.

```
        TD$ENH      = ^O2000000000          ; Define the inSight TDV flag
                                            ; for older OS's
        DEL         = ^O177                 ; Define the del character

        TRMCHR      @A4, 0            ; Load term characteristics into
                                      ; impure memory
        MOV         TC.FLG(A4), D7    ; Get terminal flags
        AND         #TD$ENH, D7 ; Check for inSight
        JNE         NORM              ;  No inSight so process normally

        MOV         #^H0FFC0, D1      ; Set up TCRT(-1, 192)
        TCRT

        TTYI                          ; Output message box lead in
        ASCIZ /B # /
        EVEN
        MOV         2(A3), A2         ; Get base of passed message
        MOV         6(A3), D0         ; Get size of passed message
10$:    MOVB        (A2)+, D1         ; Get a character
        BEQ         20$               ; Done with string
        TTY                           ; Output character
        SOB         D0, 10$           ; Loop till done

20$:    MOV         #DEL, D1          ; Caption terminator
        TTY
        LEA         A2, TITLE         ; Get the title
        TTYL                          ; Output it

        KBD                           ; Wait for response
        KBD                           ; Wait 2nd for response
        RTN

TITLE:  ASCII /OK to Continue/
        BYTE        DEL, 0
        EVEN
```

If the message routine is contained in an include file, refer to the following example:

```
MESSAGE:
! Routine to display single line message to user, message is
! contained in    ! MESSAGE.
!
      IF    ivt = 1                       &
      GOTO  insight'msg

! Normal code.

insight'msg:
      M'CAPTION'STRING  = "OK to Continue"
      M'BUTTON                = MB'OK
      M'ICON                  = MB'ICONSTOP
      M'DFLT'BUTTON           = MB'DEFBUTTON1
      M'CAPTION'STRING        = MESSAGE
      CALL                    MESSAGE'BOX
      RETURN
```

Colorization

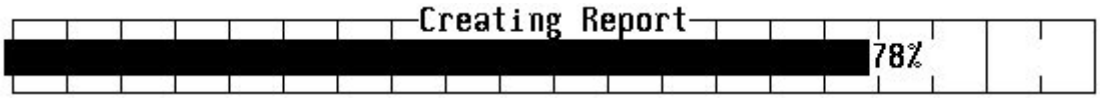
Adding color to your application can add a little pizzazz to the screens. It takes a very small amount of color to change the overall look. Try some of these tips.

- Stick to a single background color for the entire screen. Use blue or white to give the Windows look. If you use the top line for program information, you can use a complementary color for the background of this line for a little splash.
- Use a high contrast color for the normal foreground. If the background is blue use a white foreground. If the background is white use a black foreground.
- Distinguish the data from the field descriptions by using high/low intensity.
- Highlight important fields in red. For example: a past due date or a high credit balance.
- Be subtle; too much color distracts the eye from the important information.

Progress Indicators

A progress indicator is that little percentage of completion bar shown by many Windows programs. Adding a progress indicator shows the user the application is doing something during a long process. Many applications display the current key or other field within the record as it's being processed, but this creates a lot of overhead on the communications channel. This method also does nothing to indicate where in the process the program is. The progress indicator function transmits data only when necessary and shows the user the current location within the process.

To fully use a progress indicator the program must know the total number of items to process so it can know where it is within the procedure. For random files the application typically maintains the count of the active records; with ISAM the rock contains this information. Many methods exist to extract the rock from an ISAM file. Once you have the count you simply make calls to update the indicator. The provided *in*Sight INCLUDE files support both Windows (above) and terminals (below).



The most common place for this function is during report generation where the program must read an entire database to collect the appropriate information. If the database is large the time may be from many seconds to hours. The percentage of completion lets the user know the machine is still working and approximately when the procedure will finish. Here's a sample use:

```
P'MAX        = CUSMAS'CNT
P'TITLE            = "Creating Report"
FOR   REC1   = 1 TO CUSMAS'CNT
 P'CURRENT   = REC1
 CALL        PROGRESS
READ               #CUSMAS'CHA, CUSMAS
 CALL        PROCESS'RECORD
 IF   ABORT        = YES         &
      P'CURRENT    = P'MAX            : &
      CALL         PROGRESS           : &
      REC1         = CUSMAS'CNT

NEXT  REC1
```

In the preceding code the section with the IF statement is the way to close the progress indicator before the end of the items (as in an abort situation). By setting P'CURRENT equal to P'MAX and calling PROGRESS you clear the indicator from the screen.

List Boxes

Adding a list box makes selecting from a finite set of items easier. Typical uses are fields like insurance carriers, printers, procedures—anywhere you have a set of items the user selects from, especially when a selection decode or ID is used (e.g. MC for Medicare). Requiring users to memorize these IDs is often impractical.

The list box function works on both workstations and terminals. On a terminal, you need to be careful not to wipe out a portion of the screen with the list box. This is not a problem on a workstation or if the terminal supports save and restore screen areas. Products like MULTI and Tracker provide all terminals with this feature. For terminals without save and restore area support, you need to put the list box in an area without any data or that can be easily repainted. Or, you can choose not to support the list box feature on these terminals. This means you must

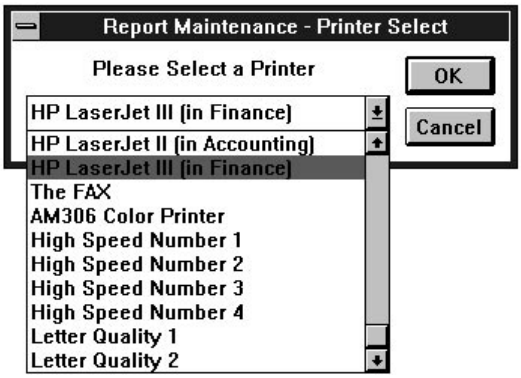
support these terminals with your own code. In other words, if you determine a terminal does not support Windows or save and restore, scroll each item on the bottom of the screen when the user enters a certain character. This works well for list selection in general. If you have a list of items present, prompt the user at the bottom of the screen to inform them of this, then use a key like TAB to select the list box. If the terminal supports Windows or save and restore, call into the LIST function; otherwise scroll through the list at the bottom of the screen as they press TAB. This adds some complexity to the code, but if you put this function in a subroutine and set up a standard calling method any program can use the feature easily.

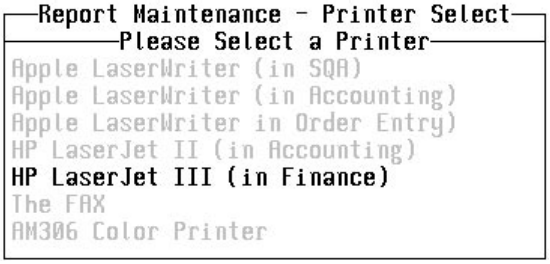
Following is an example of simple printer selection.

```
L'X                     =5
L'Y                     = 20
L'HIGH                  = 4
L'TITLE                 = "Report Maintenance - Printer Select"
L'PROMPT                = "Please Select a Printer"
L'ITEM'COUNT            = 4
L'ITEM'FLAG(01)   = " "
L'ITEM'TEXT(01)   = "Apple LaserWriter (in SQA)"
L'ITEM'FLAG(02)   = " "
L'ITEM'TEXT(02)   = "Apple LaserWriter (in Accounting)"
L'ITEM'FLAG(03)   = " "
L'ITEM'TEXT(03)   = "Apple LaserWriter in Order Entry)"
L'ITEM'FLAG(04)   = " "
L'ITEM'TEXT(04)   = "HP LaserJet II (in Accounting)"

CALL        LIST
IF    LIST'RESULT       = 0         &
      RETURN
```

One way to use list boxes in an application where the user is prompted to enter a decode, especially when the input codes are well known, is to prompt for the input code. Then, if the user presses a certain key, like TAB, the program presents a list box. That way the users who know the codes can continue to do fast data entry, while others can bring up the entire list. It is a good idea to show the decode in the L'ITEM'TEXT string to allow the user to get familiar with them.

The following two examples show both *in*Sight and non-*in*Sight style list boxes.





Mouse Enabling Your Screens

One area that excites many AMOS programmers is the ability to add mouse positioning to an old program. This is not as difficult as it may seem. It also does not require *in*Sight, as this feature is supported by the workstation itself. Mouse support also works on AM-72s and AM-75s (with the graphics card). A few methods exist for determining mouse position from within a program. We'll discuss a way that seems to be one of the easiest to retrofit into existing programs. The idea behind this method is setting up *hot spots* on the screen. You can tell the workstation or terminal to recognize areas on the screen, so when a user clicks inside one a predetermined string is sent to the host program. In other words, your program designates the area surrounding a field as a hot spot and tells the terminal to send a specific string if the user clicks the mouse on that field.

For example, many programs have numbered fields and a prompt at the bottom of the screen for field number. If your program tells the terminal to send back RETURN, field number, RETURN when a user clicks on a field, the user is taken to the field he or she selects. If the user clicks on field 2, the terminal sends RETURN (to finish input on any other field the user may be on), then 2 and RETURN, to answer the `Which field?` prompt. The only modification to the program is to load the locations of the fields when the screen is painted and remove all the hot spots when the user closes the screen. This also works well for menus.

You can also use this technique if you use up/down arrow style field movements, especially when using something like INFLD for data input. With an input editor like INFLD, you can specify which keys trigger an exit from the field. By loading the right sequence into the hot spots you can make your input routine determine if the mouse has been pressed.

As an example, by using INFLD's function key processing you can determine that an obscure function key was pressed. INFLD uses ^G as a lead-in for manual function key processing followed by a character that determines the function key. You could use ^G followed by Z, which produces an exit code of 65501. If your input routine checks for this exit code, you can jump to a special function that reads the next input character, which defines where the mouse was pressed. The fields are programmed to send back a ^G•Z•A for field 1, ^G•Z•B for field 2, ^G•Z•C for field 3 and so on. The input routine can easily determine which field was clicked.

The following example program shows a simple field selector using the mouse. The include files are shown separately.

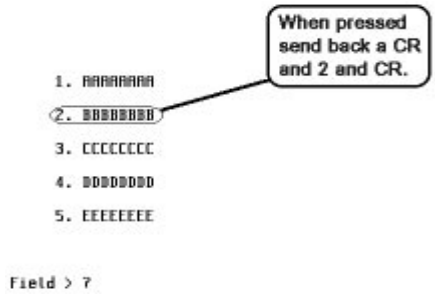
```
++INCLUDE MOUSE.MAP

         ON          ERROR GOTO EXIT

         CALL        init'hot'spot

         PRINT       TAB(-1,0);
         PRINT       TAB(08,10);"1. AAAAAAAA";
         PRINT       TAB(10,10);"2. BBBBBBBB";
         PRINT       TAB(12,10);"3. CCCCCCCC";
         PRINT       TAB(14,10);"4. DDDDDDDD";
         PRINT       TAB(16,10);"5. EEEEEEEE";

!        hot'spot(x) is formatted as follows:
!                    XXYYLLSS~SS
!        where
!                    XX    Starting row
!                    YY    Starting Column
!                    LL    Length of field
!                    SS~SS String to return
!
         hot'spot(1) = "081010"+CHR(13)+"1"+CHR(13)
         hot'spot(2) = "101010"+CHR(13)+"2"+CHR(13)
         hot'spot(3) = "121010"+CHR(13)+"3"+CHR(13)
         hot'spot(4) = "141010"+CHR(13)+"4"+CHR(13)
         hot'spot(5) = "161010"+CHR(13)+"5"+CHR(13)
         CALL        set'hot'spot

LOOP:    PRINT       TAB(20,5);"Field > ";TAB(-1,9);
         X           = 0
         INPUT       X
         IF          X = 0       &
                     GOTO  LOOP

         FOR Y       = 1 TO 5
          PRINT TAB  (Y*2+6,8);
          IF         Y = X       &
               PRINT ">";        &
             ELSE                &
               PRINT " ";

         NEXT        Y
         GOTO        LOOP

EXIT:    CALL              clear'hot'spot
         END

++INCLUDE MOUSE.BSI
```

```
MOUSE.MAP:
         MAP1        hot'spot(75)
          MAP2       hs'row      , S   , 2
          MAP2       hs'col      , S   , 2
          MAP2       hs'len      , S   , 2
          MAP2       hs'str      , S   , 10
         MAP1        hot'cnt     , F   , 6
         MAP1        hs'r        , F   , 6
         MAP1        hs'c        , F   , 6
         MAP1        hs'l        , F   , 6
         MAP1        hs'x        , F   , 6
         MAP1        hs'key      , S   , 10
         MAP1        hotspt      , F   , 6
```

```
MOUSE.BSI:
!****************
init'hot'spot:
!****************
!
! Initialize hot spot processing.
!
!  NOTE: Call once at top of program
!
! input:
!              none
! output:
!              none
!
               XCALL WINFLG, hotspt, 162.0
               RETURN

!****************
set'hot'spot:
!****************
!
! Set the hot spots for this screen
!
! input:
!              hot'spot(x) Set the hot spot table (hs'row, hs'col, hs'len).
! output:
!              none
!
               hot'cnt      = 0
               IF    hotspt = 0              &
               RETURN

               FOR   hs'x = 1 TO 75
                IF   hs'row(hs'x) = ""       &
                     hs'x  = 75                    &
                   ELSE                            &
                     hot'cnt     = hot'cnt + 1
```

```
                NEXT              hs'x

                PRINT       TAB(-1,162);CHR$(hot'cnt+32);
                IF    hot'cnt = 0             &
                RETURN

                FOR               hs'x = 1 TO hot'cnt
                 hs'r       = VAL(hs'row(hs'x))
                 hs'c       = VAL(hs'col(hs'x))
                 hs'l       = VAL(hs'len(hs'x))
                 hs'key     = hs'str(hs'x)

                 PRINT      CHR$(LEN(hs'key)+32);hs'key;CHR(hs'r+31);

                 IF   hs'c > 96                     &
                      PRINT CHR(25);CHR(hs'c-65);   &
                    ELSE                            &
                      PRINT CHR(hs'c+31);

                 PRINT      CHR(hs'r+31);

                 hs'c       = hs'c + hs'l - 1
                 IF         hs'c > 96               &
                      PRINT       CHR(25);CHR(hs'c-65);   &
                   ELSE                                   &
                      PRINT CHR(hs'c+31);

                NEXT              hs'x
                hot'cnt           = 0
                RETURN

!*****************
clear'hot'spot:
!*****************
!
! Clear the hot spots
!
! input:
!               none
! output:
!               none
!
                hot'cnt           = 0
                IF    hotspt = 0              &
                      RETURN

                FOR               hs'x = 1 TO 75
                 hs'row(hs'x)     = ""
                NEXT              hs'x
                PRINT       TAB(-1,159);
                RETURN
```

The following figure shows how the screen should look and functional points:



The Mouse and INFLD

Many developers use INFLD for entering text into their application. INFLD lends itself very well to mouse positioning. Since INFLD can accept control characters you can make distinguishing a mouse from normal input simple and almost invisible to the user. Using the methods above you can expand the effectiveness and recognition. In the above examples the mouse simply inputs what the user could input manually. Below we'll show how to use function keys not only to identify mouse use, but to make it difficult for the user to mistakenly input.

As the lead-in sequence of each hot spot you can use the ^GZ entry code. Using INFLD, ^G is the keyboard method of entering a function key. Following ^G the user enters 1-9 (F1-F9), A-G (F10-F16), H-W (sF1-sF16 or F17-F32), or X-Z (F33-F35). Obviously F35 (^GZ) is not a valid function key, but it is accessible, logically, through INFLD. This makes it a perfect candidate for the mouse lead in sequence. INFLD reports function key presses by returning EXITCODE in the range of -1 (F1) to -35 (F35) when EXITCODE is defined as a floating point variable. If you are using the PACKET method of variable passing to and from INFLD, EXITCODE is MAPped as a B,2, which means function keys are returned as 65535 (F1) to 65501 (F35).

Following the ^GZ lead in use a unique identifier for each field. You could simply use the ASCII set starting at SPACE (32) and increment for each field. Then when your input routine sees an F35 exit code the next character (minus an ASCII 31) is the field number. Using the above example we can show the modifications to the individual hot spots.

```
hot'spot(1) = "081010"+CHR(7)+"Z"+CHR(32)
hot'spot(2) = "101010"+CHR(7)+"Z"+CHR(33)
hot'spot(3) = "121010"+CHR(7)+"Z"+CHR(34)
hot'spot(4) = "141010"+CHR(7)+"Z"+CHR(35)
hot'spot(5) = "161010"+CHR(7)+"Z"+CHR(36)
```

Then immediately after determining F35 has been received do the following:

```
         IF exitcode = 65501           &
               GOTO  mouse             ! Got the F35 so get the field

mouse:   XCALL ACCPT, xxx
         field = xxx-31
         RETURN
```

By using the 'GO' command as one of the field type codes (lower case g) you do not need the carriage return as the first character returned from the mouse click. This type code tells INFLD to treat function keys like RETURN, up and down arrow, etc., and completes field processing before returning. Otherwise INFLD restores the original contents of the field the user was editing just prior to the mouse click. This way you need to store only three characters at each field position, ^GZ plus the field ID.

For a simple test, run INFLD.RUN, which is part of the INFLD release. Add g to the type codes. Enter some data and then press ^GZ. You will see EXITCODE equal -35 and ENTRY equals what you entered. If you leave off the g and pre-load ENTRY, then modify ENTRY when prompted and press ^GZ you will see the original (pre-loaded) contents of ENTRY returned. Use a TYPE of ***** or *****g, MAX of 10, MIN of 0 and OPCODE of 1.

Simple Imaging

Alpha Microsystems provides a complete imaging solution in our image.doc product line. However, this is more than some applications require. *in*Sight supports simple imaging which acts as an entry level product to the image.doc line.

*in*Sight lets you display previously stored images. These images can be stored in a number of formats, but must reside on the PC or PC network. You can display multiple images at once and can individually control basic features of the image—like rotation, contrast, brightness—once it is displayed. You can also control many features of the image before you display it, including print capabilities, user sizing, title and border display, initial orientation, initial position and size, etc. Although not a complete image management system, *in*Sight provides enough functionality to meet the demands of quite a few applications. If the application needs to display a handful of images and can rely on image storage and manipulation from another source, *in*Sight will work for you.

Fundamentally, image display through *in*Sight is a matter of providing the SHOW'BITMAP routine with an image file name, a description, some flags and the initial coordinates. Your application must maintain and provide this information. You may need to create a simple image database on AMOS to store the information. If you have a one to one relationship between a record and an image, simply adding an image file name field to your database may be all that's needed. The example shown below can associate multiple images to a single record in the database. This requires the use of a secondary file. If organized correctly, this single image file can service your entire application and link images to any one of your data files.

Creating a single image data file to service multiple data sets is fairly simple. If you look at the MAP statements for the sample data file you will see that the key (I'KEY) is 20 characters long. This should handle the longest key in most databases. Our example uses ISAM Plus, which allows for duplicate keys, which is how I'KEY is defined. If you use another database manager you need to incorporate some mechanism to handle duplicate keys since multiple images may be associated with a single data record. Let's say you want to tie images to your customers as well as your inventory. In this case the key you build for the image will be C plus the customer number or I plus the part number. This lets you quickly determine which image is associated with which data file. You can add new associations simply by changing the lead character of the I'KEY. During a lookup you just append the appropriate lead character to the main key of the data file and search.

You could further break out the images by subsection by appending a type code to the end of the key. For instance, you want to maintain an image of customer signatures, initial contract, and any correspondence. To uniquely identify one from another in the data base, append an S for signature, a C for contract or an L for letters to the end of I'KEY. For an image of a letter from customer number 10345, I'KEY would be "C10345L**"**. You can now search for any level of the key and then sequentially read data until the keys don't match. And by the way, you can use this same image file with image.doc.

Another *in*Sight routine is SHOW'BITMAP'FROM'LIST. This matches well to the above database where multiple images may be associated with a record. It allows your application to load an array of associated images and use *in*Sight's LIST function to ask the user which one to display. Once the user is done making selections you can clear all the images from the screen by using the CLEAR'BITMAPS function.

The following example shows a simple image display routine. The underlying database is ISAM Plus in this case, but, of course, could be adapted to any other database engine. To actually try this example you need to create an ISAM Plus database as follows:

```
.ismutl image [RETURN]
Alpha Micro Indexed Sequential Access Method Version 2.1(121)-1

Number of records to allocate: 20
Size of Data Record: 102

Type of key: Enter 1 for Alphanumeric type
             Enter 2 for Unsigned Binary type
             Enter 3 for Concatenated keys type
             Enter 4 for Unsigned BASIC Binary type

Primary key
Type of key: 1
Key description: Image Key
Are duplicate keys allowed for this directory index? (Y/N) Y
Are keys allowed to change during updating? (Y/N) N
Size of key: 20
```

```
Position of key: 1

Secondary key number 1
Type of key:
3 index blocks will be allocated
Enter extra index blocks percentage:
6 total index blocks will be allocated
Enter loading factor:
Data file Device:

File created
Press the RETURN key to continue
```

The image data file layout (IMAGE.MAP):

```
MAP1        IMAGE'FILNAM        , S     , 30    , "IMAGE"
MAP1        IMAGE'CHANNL        , F     , 6     , 1001
MAP1        IMAGE'RELREC        , F     , 6
MAP1        IMAGE'STAT    , F   , 6

MAP1        IMAGE               ! Image Record
 MAP2  I'KEY        , S     , 20    ! Image Key
 MAP2  I'FIL        , S     , 40    ! PC Filename of image
 MAP2  I'DES        , S     , 30    ! Description of image
 MAP2  I'FLG        , B     , 4     ! Preset flags
 MAP2  I'X          , B     , 2     ! Initial location
 MAP2  I'Y          , B
```

The following code simply adds three records to the image database:

```
++INCLUDE      IMAGE.MAP

         DATA        "C10202","c:\windows\rivets.bmp"
         DATA        "C10202","c:\windows\tartan.bmp"
         DATA        "C10203","c:\windows\thatch.bmp"

         OPEN        #IMAGE'CHANNL, IMAGE'FILNAM, INDEXED, IMAGE'RELREC, &
                     IMAGE'STAT, WAIT'FILE

         FOR               X = 1 TO 3
          READ       I'KEY,I'FIL
          I'DES             = I'FIL
          I'FLG             = 32784
          I'X        = 0
          I'Y        = 0
          CREATE'RECORD     #IMAGE'CHANNL, IMAGE
         NEXT               X

         END
```

Following is the actual lookup and display image code. The customer number is hard-coded for ease of the example, but the `show'image` routine is fairly generic and should work in a variety of environments. Simply set IMGKEY to the appropriate value and call `show'image`.

```
++INCLUDE      IVTERM.MAP
++INCLUDE      IVTBIT.MAP
++INCLUDE      IMAGE.MAP

         MAP1        IMGKEY      , S   , 20

         MAP1        CUSNUM      , S   , 5   , "10202"   ! Customer number

         CALL        IVTERM'INIT
         OPEN        #IMAGE'CHANNL, IMAGE'FILNAM, INDEXED, IMAGE'RELREC, &
                     IMAGE'STAT, WAIT'FILE

         IMGKEY      = "C" + CUSNUM
         CALL        show'image

         END

!************
show'image:
!************
! Allow user to select an image to display from the list defined by the
! search key.
!
! Input:
!        IMGKEY      The search key
!
         GET         #IMAGE'CHANNL, ISAM'KEY(0) = IMGKEY, IMAGE
         IF    IMAGE'STAT = -3               &
               RETURN

         BL'COUNT                = 0
si001:   BL'COUNT                = BL'COUNT + 1
         BL'FILNAM(BL'COUNT)     = I'FIL
         BL'FLAGS(BL'COUNT)      = I'FLG
         BL'X(BL'COUNT)          = I'X
         BL'Y(BL'COUNT)          = I'Y
         BL'DESC(BL'COUNT) = I'DES

         GET'NEXT    #IMAGE'CHANNL, IMAGE
         IF    IMAGE'STAT = 0          &
                   AND                 &
               I'KEY = IMGKEY          &
             THEN                      &
               GOTO  si001
```

```
si002:   CALL        SHOW'BITMAP'FROM'LIST
         IF    BL'RESULT # 0                 &
             AND                             &
               BL'COUNT > 1                  &
             THEN                      &
               GOTO  si002

         M'BUTTON          = MB'OK
         M'ICON            = 0
         M'DFLT'BUTTON     = MB'DEFBUTTON1
         M'TEXT'STRING     = ""
         M'CAPTION'STRING= ""
         CALL        MESSAGE'BOX
         CALL        CLEAR'BITMAPS
         RETURN

++INCLUDE IVTERM.BSI
++INCLUDE IVTBIT.BSI
```

When you run the above program you should see the following dialog box. If you double click on one of the images the dialog box should disappear and the image should be displayed. Because of the loop (`si002`), this process will be continually repeated until the user selects *Cancel*. At that point a simple "click to continue" message will appear via MESSAGE'BOX**.** Once it is clicked, any images will be cleared from the screen by CLEAR'BITMAPS.

Help System

*in*Sight gives you the ability to display a help message based on the capabilities of the user's terminal or workstation. Using *in*Sight's help and associated AMOS routines you get a comprehensive help system that works across any of the terminal types and includes access into the Windows help system. You can create a Windows help file for your product. By setting up the appropriate functions you can provide context-sensitive entry into Windows help.

The fundamental approach of the *in*Sight help system is that for every piece of help available there is an associated help file on the AMOS system. Each of these files contains information on the topic. There is a single line help message, a multi-line message, and, if available, a pointer to the Windows help file. With this information any terminal is covered.

The help system provides for four possibilities:

- An inSight workstation with a Windows help file is launched into Windows help, directly to the topic of choice.
- An inSight workstation with no Windows help file gets a Windows message box with the multi-line help message.
- A terminal that supports save and restore screen areas gets a multi-line help message displayed in a window on the screen.
- A terminal with no save/restore gets a single help line at the bottom of the screen.

This makes sure the contents of the screen prior to the help display are preserved.

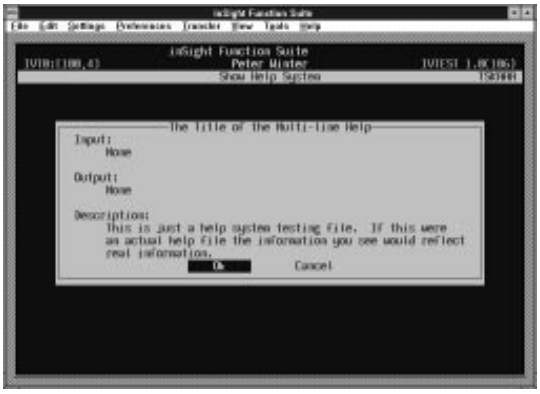
The following example shows what the contents of a help file look like. This is a straight AMOS text file:

```
Single=This is a single line help message
Windows=winorbit.hlp,host
Multi=The Title of the Multi-line Help
Input:
     None

Output:
     None

Description:
     This is just a help system testing file.  If this were
     an actual help file the information you see would reflect
     real information.
```

In the above case we've defined a Windows help file. This will invoke WINORBIT.HLP and search for the host topic in it. The title of the multi-line help display window immediately follows the equal sign and the rest of the text to the end of the file is the help message. You can define multiple pages of data by placing a **\p** in the multi-line help (optionally followed by a new title.) To access this help file we need to create the following incredibly complex program:

```
++INCLUDE IVTERM.MAP
++INCLUDE IVTHLP.MAP

         CALL        IVTERM'INIT

         HELP'FILNAM = "BAS:IVTEST.HLP"
         CALL              LAUNCH'HELP
         END

++INCLUDE IVTERM
++INCLUDE IVTHLP
```

Here are examples of a couple of the forms of help. The left shows an *in*Sight workstation with a Windows help file. The right shows a terminal that supports save and restore screen areas.



Although *in*Sight provides direct access into the Windows help system, if you want to take advantage of Windows help you will need to create the Windows help file. Windows help is created through a variety of methods. At the lowest level is a help compiler which takes Rich Text Files as input and produces the actual file you access through WINHELP.EXE. You can create these RTF files using a word processor like Word. Tools, like RoboHelp, which link to Word make the whole process easier. Unfortunately, documentation created in the AMOS help files cannot be converted to Windows help automatically. If you wish to support both Windows help and existing standard terminals, you need to maintain both copies of the help system, one on AMOS and one in Windows help.

Launching Windows Programs

In order to show even further integration with Windows, your application can launch any Windows program available on the workstation. For instance, you may want to launch the user into an image editor for one of the bitmaps associated with your application. Once your application has determined which image from the AMOS database to modify, you invoke the image editor. Since most Windows applications accept command line arguments, such as the filename to process, you append the filename to the image editor and have *in*Sight launch the entire thing. The user is automatically brought into the editor with the image they selected from the AMOS database.

This is only one example of the use of the launcher. Through the file I/O commands you could create a mail merge file from your AMOS database directly to the PC. You could then launch a word processor with a template file that understood the mail merge data layout. Of course many variations of this exist: spreadsheets, databases, CAD, etc.

So let's take a look at how it's done. Since IVT'LAUNCH is part of IVTERM.BSI, that, and IVTERM.MAP, are the only include files necessary. The example that follows assumes the image file name was extracted from an AMOS database rather than simply being hard coded:

```
++INCLUDE IVTERM.MAP
         MAP1              IMAGE       , S   , 50
         CALL              IVTERM'INIT

         IMAGE       =  "C:\WINDOWS\REDBRICK.BMP"
         CALL              EDIT'IMAGE
         END

EDIT'IMAGE:
         IL'FLAG           = 4         ! Launch program maximized
         IL'COMAND         = "PBRUSH.EXE "+IMAGE
         CALL              IVT'LAUNCH
         RETURN

++INCLUDE IVTERM.BSI
```

Private Profile Access

The private profile file is Windows' way to store information about a user's session. For example, WIN.INI and SYSTEM.INI are private profile files used by Windows itself. A private profile file typically stores information about the user's preferences, last known screen position, etc., but can store any type of information. *in*Sight provides functions to both read and write these files. In general a private profile file is divided into sections and subsections (entries). Information is stored at the subsection level. If, for instance, you want to store the user's name and experience level for each workstation, you can create a private profile file on the PC. Within the file you can set up a *User Info* section and in that store the name under *Name* and the experience level under *Exp Level*. There is no limit to the number of sections and subsections within a single private profile.

Since most Windows applications use this style of storage, you could inquire and/or change a different application's parameters so when the user next accessed it, it could match parameters in your application or your application could match its. Or, since AlphaNET connections don't define which physical workstation is attached to a job, you could save a profile on each PC that defined that user. An option on your program could prompt for the user name and then update a profile on the PC for later retrieval. You could also use the profile to store such things as which word processor is on this workstation, if they have a spreadsheet, etc.

You can store three types of data within a profile file; string, integer and Boolean (T/F). Two commands, one for reading and one for writing, exist for each data type. The commands are:

read'profile'string	Read a string
read'profile'int	Read an integer
read'profile'bool	Read a Boolean value
write'profile'string	Write a string
write'profile'int	Write an integer
write'profile'bool	Write a Boolean value

Three pieces of information are required to read a profile argument: the profile file name, the section, and the subsection. You also need to know what type of argument it is. For writing you also have to supply the argument value. The following shows how you could read the user's name, change it using *in*Sight's EDIT, and then rewrite it to the PC:

```
++INCLUDE IVTERM.MAP
++INCLUDE IVTPRO.MAP

         CALL        IVTERM'INIT
         CALL        CLEAR'SCREEN

         p'filnam    = "C:\MYAPP\MYAPP.INI"
         p'section   = "User Info"
         p'entry     = "Name"
         CALL        read'profile'string
         E'TITLE     = "Change User Name"
         E'PROMPT    = "Enter your user name"
         E'TEXT      = p'strval
         CALL        EDIT
         p'strval    = E'TEXT
         CALL        write'profile'string
         END

++INCLUDE IVTERM.BSI
++INCLUDE IVTPRO.BSI
```

File I/O

The file I/O functions provide the tools to read and write standard text files to and from the PC. You can also use standard file selection dialogs to determine which PC file to access. The file selection function is also valuable when used with the imaging functions to select an image file from the PC.

Some possible uses for the File I/O functions are:

- Creating a file on the PC from your application database for importing into something like Excel or Paradox. Once you've created the file on the PC, use the *in*Sight launch facility to automatically bring the user into the spreadsheet.
- Create a mail merge data file for Word or another word processor.
- Some third party applications look for a specific data file when launched to retrieve configuration information or initial record identifiers. You could build the file in the application's format and then launch the application.

The following example shows how you could display a PC file selected by the user:

```
!*********************************************************************
! Example of file read routines
! NOTE: Load ACCPT.SBR for improved performance
!*********************************************************************
++INCLUDE ivterm.map
++INCLUDE ivtfil.map

      CALL      IVTERM'INIT

! Set up the select file dialog function
      FI'FILNAM     = "c:\insight\"         ! Default directory
      FI'FILCNT     = 2                     ! 2 file filters
      FF'DESC(1)    = "Initialization Files (*.INI)"  1st are .INI files.
      FF'WLDCRD(1)  = "*.INI"               ! The wildcard spec
      FF'DESC(2)    = "Text files (*.TXT)"  ! 2nd are Text files
      FF'WLDCRD(2)  = "*.TXT"               ! The wildcard spec
      FI'TITLE      = "Select a file to read"
                                            ! Dialog title
      CALL          GET'FILENAME            ! Call the inSight
                                            ! function
      IF        FI'FILNAM = ""     &
                END

! Set up to open the file for reading
      FI'ACTION     = FA'READ               ! File action is read
      CALL          OPEN'FILE               ! Call inSight open
                                            ! function
      PRINT     "Open Result :";FI'RESULT   ! Print the open result status
      IF        FI'RESULT &
                END

! Read the file, line by line
      LIN                = 0                ! Simple line counter
      LOOP:     CALL     READ'FILE          ! Read a record
      PRINT              FI'RESULT;"> ";    ! Print read result
                                            ! status
  IF  FI'RESULT     &
      CALL CLOSE'FILE    : &
      END

  LIN           = LIN + 1                       ! Increment the line
                                                ! count

  PRINT         (LIN USING "##");". ";FI'RECORD
                                                ! Print line number and
                                                ! record
  GOTO          LOOP                            ! Do it some more

++INCLUDE ivterm
++INCLUDE ivtfil
```

CHAPTER SEVEN
WRITING AN INSIGHT DLL CALL

This chapter describes the procedures you need to follow to write a Windows DLL function to be called by the *in*Sight tool kit. In order to work correctly when called by *in*Sight, three areas of the DLL—type definitions, the function declaration, and the method of sending data back to the host program—must conform to the information given here.

This portion assumes a detailed knowledge of DLL writing under the Windows 3.1 operating system. No information concerning the actual methods and ideas is included.

DEFINING TYPES

You must include these two type definitions in the DLL or an included header file:

```
typedef     unsigned char __far *LPTEXT;
typedef     BOOL (CALLBACK *PFSEND) (HWND, LPINT, LPTEXT);
```

The ***LPTEXT*** declaration is used for all data transfer to and from the *in*Sight tool kit. It is simply a long pointer to unsigned character data.

The ***PFSEND*** declaration is used when describing the variable which passes the address of the SendRoutine function. SendRoutine lets you send data back to the host application.

DECLARING THE FUNCTION

You must use the following function declaration when creating a function to be called by the *in*Sight tool kit:

```
void FAR PASCAL _export FunctionName (HANDLE hWnd,WORD DataLen,
            LPTEXT Data,PFSEND SendRoutine)
```

FunctionName is the name of your function.

hWnd is the main Terminal Emulation Window handle. You can use this to gain access to and control of the parent window.

DataLen is the number of bytes contained at Data.

Data is a long pointer to the data passed from the application program.

SendRoutine is a long pointer to the send routine. This function is used when transmitting data back to the host. See the next section.

SENDING DATA BACK TO THE HOST PROGRAM

Your DLL function can send data back to the host *in*Sight application by calling the *in*Sight DLL's SendRoutine function. Here is a description of that function, in approximately the same format used by *Microsoft Windows 3.1 Programmer's Reference*:

```
BOOL (CALLBACK *SendRoutine) (hWnd, lpCnt, lpText)
HWND        hWnd;   /* Handle of parent window */
LPINT       lpCnt;  /* Address of the int containing the number of bytes
                        to send */
LPTEXT      lpText; /* Address of the byte stream to send*/
```

The ***SendRoutine*** function transmits the contents of ***lpText*** to the host application. The data sent looks like simple data input to the host application (as though it came from the keyboard.) You can make this call any number of times within one iteration of the called function.

Parameters

hWnd identifies the handle of the parent window. This value is sent as the first parameter of the initial call from *in*Sight.

lpCnt points to an integer that contains the total count of the bytes to transfer to the host. The value at lpCnt will be decremented to zero within the SendRoutine function.

lpText points to a block of unsigned char's that will be transmitted to the host.

Return Value

SendRoutine returns 1 if data was successfully transmitted to the host; otherwise it returns 0.

EXAMPLE

```
int SCnt;

SCnt= 11;
SendRoutine(hWnd, (LPINT)&SCnt,(LPTEXT)"0987654321\r");
```

Sample DLL definition file

```
LIBRARY           TESTDLL

DESCRIPTION       'DLL for testing the DLL call functions of inSight'
EXETYPE           WINDOWS
CODE              PRELOAD MOVEABLE DISCARDABLE
DATA              PRELOAD MOVEABLE SINGLE
HEAPSIZE          1024
```

Sample DLL header file

```
typedef  unsigned char TEXT;
typedef  unsigned char *PTEXT;
typedef  unsigned char __far *LPTEXT;
typedef  unsigned char __far *(__far *LPPTEXT);
typedef  const unsigned char *PCTEXT;
typedef  const unsigned char __far *LPCTEXT;
typedef  const unsigned char __far *(__far *LPPCTEXT);
typedef  unsigned long ULONG;
typedef  const char *PCSTR;
typedef  BOOL (CALLBACK *PFSEND) (HWND, LPINT, LPTEXT);
extern void FAR PASCAL _export TestFunction(HWND, WORD, LPTEXT, PFSEND);
```

Sample DLL program file

```
#include <windows.h>
#include "testdll.h"

extern int FAR PASCAL _export WEP (int);

/*
 * Required DLL prolog
 */
int FAR PASCAL LibMain (HANDLE hInstance, WORD wDataSeg,
                            WORD wHeapSize, LPSTR lpszCmdLine)
{
if (wHeapSize > 0 )
            UnlockData(0);

return 1;
}

int FAR PASCAL _export WEP (int nParam)
{
return 1;
}

/*
      * Sample DLL function
      *
* This function simply takes the passed data and uses it as the message
* for a MessageBox.  The return string (which is optional) is hardcoded
* as a simple example of how to return data to the host.
```

```
*

void FAR PASCAL _export TestFunction (HANDLE hWnd, WORD DataLen,
                                          LPTEXT Data, PFSEND
SendRoutine)
{
TEXT        work[500];
LPTEXT      wpnt;
int         Scnt;

            if( DataLen > 499 )
                  DataLen = 499;

            wpnt  = (LPTEXT) work;

            while( DataLen-- )
                  *wpnt++ = *Data++;

            *wpnt = NULL;

            MessageBox(hWnd,work,"Inside the Test DLL Function", MB_OK);

            SCnt  = 11;
            SendRoutine(hWnd, (LPINT)&SCnt, (LPTEXT)"0987654321\r");

            return;
}
```

APPENDIX A
INSIGHT SOFTWARE: FILE LISTING

The following table lists the *in*Sight release files and their functions.

FILENAME/LOCATION	FILE DESCRIPTION
IEXIT.LIT[1,4]	AMOS level program to terminate the Terminal Emulator.
IVTCAP[1,4	AMOS level program to change the caption of the window.
IVTVER[1,4]	AMOS level program to place the version into the window title.
IVTBIT.BAS[7,6]	Sample AlphaBASIC program to do image display.
IVTDLL.BAS[7,6]	Sample AlphaBASIC program to access a custom Windows DLL.
IVTEST.BAS[7,6]	Sample AlphaBASIC program to show the majority of the *in*Sight functions.
IVTFIL.BAS[7,6]	Sample AlphaBASIC program to show file I/O.
IVTNAM.BAS[7,6]	Sample AlphaBASIC program to request user filename.
IVTPRO.BAS[7,6]	Sample AlphaBASIC program to show profile access.
MOUSE.BAS[7,6]	Sample AlphaBASIC program to show mouse support.
IVTBIT.MAP&.BSI[7,6]	Support code for image functions.
IVTERM.MAP&.BSI[7,6]	Support code for general *in*Sight functions.
IVTFIL.MAP&.BSI[7,6]	Support code for file functions.
IVTHLP.MAP&.BSI[7,6]	Support code for help functions.
IVTPRO.MAP&.BSI[7,6]	Support code for profile functions.
JOBDAT.MAP[7,6]	Support maps for the JOBDAT XCALL routine.

FILENAME/LOCATION	FILE DESCRIPTION
*.RUN[7,6]	Compiled versions of sample programs (AlphaBASIC 1.4).
*.ORN[7,6]	Compiled versions of sample programs (AlphaBASIC 1.3).
ACCPT.SBR[7,6]	AlphaBASIC XCALL to return a single character from the keyboard.
EDIT.SBR[7,6]	Supporting XCALL for *in*Sight.
FSTRIP.SBR[7,6]	AlphaBASIC XCALL to strip leading and trailing spaces from a string.
JOBDAT.SBR[7,6]	AlphaBASIC XCALL to return information about the attached job.
LIST.SBR[7,6]	Supporting XCALL for *in*Sight.
MENU.SBR[7,6]	Supporting XCALL for *in*Sight.
MSGBOX.SBR[7,6]	Supporting XCALL for *in*Sight.
WINFLG.SBR[7,6]	AlphaBASIC XCALL to determine terminal characteristics.
*.VBR[7,6]	AlphaACCOUNTING replacement subroutines. To use these XCALLs, see Chapter 2.
.[7,35]	AlphaBASIC+ equivalents to the [7,6] account.

This file listing is accurate at the time of this document's writing; however, the files and their locations are subject to last minute changes due to release requirements.

APPENDIX B
MODIFYING TERMINAL DRIVERS

The *in*Sight for AMOS software includes two terminal drivers which support the *in*Sight subroutines, one for mode terminals and one for field terminals. These terminal drivers use a "native" mode emulation on the workstation. If you want to use the *in*Sight routines while using a different terminal emulation on the workstation, you need to modify the appropriate terminal driver. This appendix tells you how to do so.

Only use the modified terminal driver with a workstation running the appropriate terminal emulation. Maintain an unmodified driver to use with "real" terminals. Using the modified driver on a non-GUI terminal could result in garbage being displayed on the screen.

As an example, we'll use the standard Alpha Micro AM-62A terminal driver, AM62A.M68. Here are the parts of the terminal driver which need to be modified: text you need to add for *in*Sight support is in bold; footnotes following the example explain what each change is for.

```
    SEARCH    SYS
    SEARCH    SYSSYM
    SEARCH    TRM

    OBJNAM    .TDV

    IF NDF,TD$ENH,TD$ENH    = ^O2000000000[1]

;Define capability flags
;

TDVFLG=TD$LID!TD$CID!TD$RVA!TD$DIM!TD$BLN!TD$UND!TD$EOS!TD$EOL!TD
$MLT
TDVFLG=TDVFLG!TD$SPL!TD$STS!TD$AMS!TD$KID!TD$PRT!TD$132
TDVFLG=TDVFLG!TD$SMT!TD$BOX!TD$BLF!TD$ENH[2]

;*******************
;*   AM62A        *
;*******************
;Terminal driver communications area
AM62:    WORD   TD$NEW!TD$TCH     ; terminal attributes
         BR     JMPINP            ; input routine
         RTN                      ; output routine
         BR     ECHO              ; echo routine
         BR     JMPCRT            ; crt control
         BR     JMPINI            ; INIT routine
```

```
         WORD   2.                ; impure area of one word (2
bytes)
ROWCNT:  BYTE   24.               ; number of rows
         BYTE   80.               ; number of columns
         LWORD  TDVFLG            ; terminal flags
         BR     JMPTCH            ; entry for TRMCHR
-------------------------------------------------------------

;Define feature bitmap
TCHBMP:  BYTE ^B11111111          ; 0 - 7
         BYTE ^B11111111          ; 8 - 15
         BYTE ^B11111111          ; 16 - 23
         BYTE ^B11111001          ; 24 - 31    (no horiz or vert
pos)
         BYTE ^B11111111          ; 32 - 39
         BYTE ^B11111111          ; 40 - 47
         BYTE ^B11111111          ; 48 - 55
         BYTE ^B11100011          ; 56 - 63    (no vertical
split)
         BYTE ^B11111111          ; 64 - 71
         BYTE ^B11111111          ; 72 - 79
         BYTE ^B00001111          ; 80 - 87    (no alternate
page)
         BYTE ^B11111111          ; 88 - 95
         BYTE ^B11111111          ; 96 - 103
         BYTE ^B00000011          ; 104 - 111  (no non-space
attributes)
         BYTE ^B00000000          ; 112 - 119  (no non-space
attributes)
         BYTE ^B00001111          ; 120 - 127  (spare)
         BYTE ^B00001111          ; 128 - 135  (no AM-70 style
color)
         BYTE ^B00000000          ; 136 - 143
         BYTE ^B00000000          ; 144 - 151
         BYTE ^B00000000          ; 152 - 159
         BYTE ^B00000000          ; 160 - 167
         BYTE ^B00000000          ; 168 - 175
         BYTE ^B00000000          ; 176 - 183
         BYTE ^B00000000          ; 184 - 191
         BYTE ^B00000001          ; 192 - 199
         BYTE ^B00000000          ; 200 - 207
         BYTE ^B00000000          ; 208 - 215
         BYTE ^B00000000          ; 216 - 223
         BYTE ^B00000000          ; 224 - 231
         BYTE ^B00000000          ; 232 - 239
         BYTE ^B00000000          ; 240 - 247
         BYTE ^B00000000          ; 248 - 255
-------------------------------------------------------------

;Byte offset and data tables follow for all commands
;
CRCA:
    OFFTAB    C1,C2,C3,C4,C5,C6,C7,C8,C9,C10
```

```
    OFFTAB    C11,C12,C13,C14,C15,C16,C17,C18,C19,C20
    OFFTAB    C21,C22,C23,C24,C25,C26,C27,C28,C29,C30
    OFFTAB    C31,C32,C33,C34,C35,C36,C37,C38,C39,C40
    OFFTAB    C41,C42,C43,C44,C45,C46,C47,C48,C49,C50
    OFFTAB    C51,C52,C53,C54,C55,C56,C57,C58,C59,C60
    OFFTAB    C61,C62,C63,C64,C65,C66,C67,C68,C69,C70
    OFFTAB    C71,C72,C73,C74,C75,C76,C77,C78,C79,C80
    OFFTAB    C81,C82,C83,C84,C85,C86,C87,C88,C89,C90
    OFFTAB    C91,C92,C93,C94,C95,C96,C97,C98,C99,C100
    OFFTAB    C101,C102,C103,C104,C105,C106,C107,C108,C109,C110
    OFFTAB    C111,C112,C113,C114,C115,C116,C117,C118,C119,C120
    OFFTAB    C121,C122,C123,C124,C125,C126,C127,C128,C129,C130
    OFFTAB    C131,C132,C133,C134,C135,C136,C137,C138,C139,C140
    OFFTAB    C141,C142,C143,C144,C145,C146,C147,C148,C149,C150
    OFFTAB    C151,C152,C153,C154,C155,C156,C157,C158,C159,C160
    OFFTAB    C161,C162,C163,C164,C165,C166,C167,C168,C169,C170
    OFFTAB    C171,C172,C173,C174,C175,C176,C177,C178,C179,C180
    OFFTAB    C181,C182,C183,C184,C185,C186,C187,C188,C189,C190
    OFFTAB    C191,C192,C193,C194,C195,C196,C197,C198,C199,C200
```

4

```
CRCB:

PAGE
;CRT code tables
;

C1: BYTE 30.,0                  ; cursor home (move to column 1,1)
C2: BYTE 200+13.,0              ; cursor return (move to column 1)
 \        \
 /        /
C145:                           ; select yellow reverse text
C146:                           ; select green reverse text
C147:    BYTE 0                 ; select cyan reverse text
C148:
C149:
C150:
C151:
C152:
C153:
C154:
C155:
C156:
C157:
C158:
C159:
C160:
C161:
C162:
C163:
C164:
C165:
C166:
C167:
```

```
C168:
C169:
C170:
C171:
C172:
C173:
C174:
C175:
C176:
C177:
C178:
C179:
C180:
C181:
C182:
C183:
C184:
C185:
C186:
C187:
C188:
C189:
C190:
C191:    BYTE 0
C192:    BYTE 33,'D,0
C193:
C194:
C195:
C196:
C197:
C198:
C199:
C200:    BYTE 0[5]

     EVEN

     END
```

The five additions you need to make to the terminal driver are:

1. Define the new feature flag, TD$ENH. The definition is in this format to ensure compatibility with the latest SYSSYM.

2. Add the new feature to the terminal capabilities longword.

3. Set the TAB(-1,192) flag to true.

4. Extend the entries in the escape code table through 192. In the example, we have extended it through 200, but only 192 is necessary.

5. Define the additional escape codes through 192. Again, we have defined codes through 200. Define TAB(-1,192) as ESCAPE D.

INDEX

A

Access a progress bar (Direct), 4-3
ACCPT.SBR, 4-5
AlphaACCOUNTING subroutines, 5-1
    installing, 2-1, 2-2

B

Bitmap image. *See Image display.*

C

Calling methods, 4-1
CAPTION (AlphaBASIC), 4-7
Caption change function, 6-4
Caption line, 4-7, 4-8, 4-32
Carriage return to continue, 6-6
Change caption line (Direct), 4-8
Change cursor shape (Direct), 4-9
Character input, 4-5
CLEAR'BITMAPS (AlphaBASIC), 4-10
CLEAR'SCREEN (AlphaBASIC), 4-11
Client-server architecture, 1-2
Close a file (Direct), 4-12
CLOSE'FILE (AlphaBASIC), 4-13
Color, 6-7
Color mapping, 6-2
Conventions used in this document, 1-2

D

del character, 6-6
Delete key
    in direct calling method, 4-2
Demonstration software, 2-1, 2-2
Direct calling method, 4-1
Display a list box (Direct), 4-14
Display a menu (Direct), 4-15
Display a message box (Direct), 4-16
Display an edit box (Direct), 4-17
Display an extended edit box (Direct), 4-18
DLL
    calling into, 4-46
    freeing, 4-22
    loading, 4-45
    writing, 7-1
DLL functions, 3-6
Dynamic Link Library. *See DLL.*

E

EDIT (AlphaBASIC), 4-19
Edit box, 4-17, 4-18, 4-19
Environment functions, 3-2
Escape sequences
    for direct calling method, 4-1
Ethernet requirements, 1-2
Exit terminal emulation window (Direct), 4-21
Exiting terminal emulation window, 4-33
Exiting the emulation window, 4-27

F

File access, 4-12, 4-13, 4-24, 4-25, 4-28, 4-55, 4-59, 4-61, 4-77, 4-79, 6-23
File I/O functions, 3-4
Free a DLL (Direct), 4-22
FSTRIP.SBR, 4-23
Functions. *See Function name or category.*
    calling methods, 4-1

G

General description, 1-1
Get a file name (Direct), 4-24
GET'FILENAME (AlphaBASIC), 4-25
Graphics conventions, 1-2

H

Hardware requirements, 1-2
Help system, 4-40, 4-41, 6-19
Help system functions, 3-5
Hot spot, 6-10

I

Icon, 6-2
IEXIT (AMOS), 4-27
Image, 6-2
Image database, 6-15
Image display, 4-10, 4-51, 4-53, 4-71, 4-73, 6-15
Image display functions, 3-4

image.doc product, 6-1
INFLD, 6-10, 6-14
Initialization, 4-34
Initiate action against a file (Direct), 4-28
Input functions, 3-3
*in*Sight help system functions, 3-5
INSIGHT.INI, 4-69
Installation, AlphaACCOUNTING subroutines, 2-1, 2-2
ivt variable, 6-3
IVT'INFO (AlphaBASIC), 4-30
IVT'LAUNCH (AlphaBASIC), 4-31
IVTCAP (AMOS), 4-32
IVTERM'CLOSE (AlphaBASIC), 4-33
IVTERM'INIT (AlphaBASIC), 4-34
IVTERM.BSI, 6-3
IVTERM.MAP, 6-3
IVTVER (AMOS), 4-35

J

Job name., 4-36
JOBDAT.SBR, 4-36

L

Launch a workstation program (Direct), 4-39
Launch Windows help system (Direct), 4-40
LAUNCH'HELP (AlphaBASIC), 4-41
Launching a workstation program, 4-31
Launching workstation programs, 6-21
LIST (AlphaBASIC), 4-43
List box, 4-14, 4-43, 6-8
Load a DLL (Direct), 4-45
Login, 4-36
Logo, 6-2
LPTEXT, 7-1, 7-2, 7-3

M

Make a call into a DLL (Direct), 4-46
Menu, 4-15
MENU (AlphaBASIC), 4-47, 6-5
Menu style, 6-4
MESAG.SBR, 6-6
Message box, 4-16
MESSAGE'BOX (AlphaBASIC), 4-49
Modify bitmap image (Direct), 4-53
MODIFY'BITMAP (AlphaBASIC), 4-51
Mouse cursor, 4-9
Mouse positioning, 6-10, 6-14

O

OPEN'FILE (AlphaBASIC), 4-55
Opening a file, 4-28

P

PFSEND, 7-1, 7-3
Private profile file, 4-62, 4-63, 4-64, 4-65, 4-80, 4-81, 4-82, 6-22
Private profile functions, 3-5
Program execution functions, 3-4
Program name, 4-36
Program version, 4-36
PROGRESS (AlphaBASIC), 4-57
Progress bar. *See Progress indicator*.
Progress indicator, 4-3, 6-7
Progress indicator functions, 3-3

R

Read or re-read a record (Direct), 4-59
Read or write data in a private profile file (Direct), 4-65
READ'FILE (AlphaBASIC), 4-60
read'profile'bool (AlphaBASIC), 4-62
read'profile'int (AlphaBASIC), 4-63
read'profile'string (AlphaBASIC), 4-64
Requirements, 1-2
Retrieve workstation information, 4-67
Return to continue, 6-6

S

Setup functions, 3-1
Show bitmap image, 4-73
SHOW'BITMAP (AlphaBASIC), 4-68
SHOW'BITMAP'FROM'LIST (AlphaBASIC), 4-71
Software installation, 2-1, 2-2
Software requirements, 1-2
Spaces
  stripping, 4-23
START.CMD file, 6-1
Status line caption, 4-7
System name, 4-36

T

TAB(-1,192)
  in direct calling method, 4-1
TD$ENH flag, 6-4
Terminal attributes, 4-75
Terminal drivers, 1
Terminal flags, 4-36

U

User Defined Keys command, 6-2
User interface input functions, 3-3
User name, 4-36

V

Version, 4-35

W

Windows profile access functions, 3-5
WINFLG.SBR, 4-75
Workstation display
    retrieving information about, 4-30, 4-67
Workstation environment functions, 3-2
Write or rewrite a record, 4-77
WRITE'FILE (AlphaBASIC), 4-79
write'profile'bool (AlphaBASIC), 4-80
write'profile'int (AlphaBASIC), 4-81
write'profile'string (AlphaBASIC), 4-82
Writing DLL functions, 7-1

X

XEDIT (AlphaBASIC), 4-19